EEML 2026

Lecture Notes

Asen Popov

Cetinje, Montenegro
27 July to 1 August 2026

Contents

1 Overview

These are summary notes from EEML 2026, the Eastern European Machine Learning Summer School, held in Cetinje, Montenegro from 27 July to 1 August 2026. They are built from the slide decks distributed during the week, photographs of the two lectures where no deck was given, four tutorial notebooks, and two files of personal notes. They are not a transcript. Each lecture chapter states what the lecture is taken to be arguing, gives the three to six ideas that carry that argument with the mathematics where the mathematics is the point, and closes by locating the lecture among the others.

Everything substantive traces back to the materials. Connective reasoning that does not come from a lecture is confined to marked boxes like the one at the end of this chapter, so it is always visible which claims belong to a speaker and which do not.

The week

Day	Chapter	Lecture and speaker	Source
Mon 27 Jul	2	Introduction to Deep Learning, Sarath Chandar	photos
	3	Representational Geometry and Alignment, Phillip Isola	deck
	4	Diffusion Models, Sander Dieleman	deck
Tue 28 Jul	5	Introduction to Reinforcement Learning, Andreea Deac	slides
	6	Statistics: Learning to Estimate, Kyunghyun Cho	deck
Wed 29 Jul	7	Continual Learning, Clare Lyle	deck
	8	To Be Figured Out, Razvan Pascanu	deck
Thu 30 Jul	9	ML for Mathematics, Matija Tapušković	deck
Fri 31 Jul	10	Spectral Analysis of DAGs, Isidora Stanković	deck
	11	Computationally-Efficient Learning, Dan Alistarh	photos
Sat 1 Aug	12	Geometric Deep Learning, Petar Veličković	deck

Four tutorial notebooks accompanied the week, on reinforcement learning, multimodal learning, mechanistic interpretability and model quantization. They are not written up here, since the notebooks are self-contained and are better worked through directly. Where a lecture chapter has a matching tutorial, it says so.

Where to start

On a return to this material after some time, the three synthesis chapters at the end are the point of the exercise and the lecture chapters are their evidence. Chapter 13 identifies the six ideas that recurred, including a genuine disagreement between Lyle and Pascanu about how serious the i.i.d. assumption is. Chapter 14 separates problems the speakers flagged as open from directions inferred here. Chapter 15 is an ordered queue of specific items to return to, with a reason for each.

> **NOT FROM THE LECTURE**
>
> This box is the mechanism for marking editorial additions, so here is one about the notes themselves. The two lectures with no distributed deck, Chapters 2 and 11, are reconstructed entirely from photographs of projected slides, and although the transcriptions were made at full resolution and are legible, those two chapters rest on a single source with nothing to corroborate them. The lectures with both a deck and photographs are the best evidenced, and there the photographs were left untranscribed because the deck supersedes them.

Part I

Lectures

2 Introduction to Deep Learning

Sarath Chandar • *Polytechnique Montréal and Mila*

The lecture is organised around a single asymmetry: we have known since the early 1990s what a neural network *can* represent, and we still do not know what we can reliably *train*. Chandar spends the first half establishing the representational result, the universal approximation theorem, in order to set it aside, and the second half on the obstructions that survive it: gradients that vanish through depth, initialisations that decide whether a network trains at all, and finally the independence assumption buried in every objective on the slides. The closing frame is the argument of the whole summer school in one line: current practice optimises the i.i.d. case, while "true learning" is sequential, online and continual.

Key ideas

The gradient that cancels. The derivation that opens the lecture is a single sigmoid unit, $y^{(n)} = \sigma(a^{(n)})$ with $a^{(n)} = \mathbf{w}^\top \mathbf{x}^{(n)}$, trained on the cross-entropy objective

$$\min_{\mathbf{w}} \; -\sum_{n=1}^{N} \Big[t^{(n)} \ln y^{(n)} + \big(1 - t^{(n)}\big) \ln\big(1 - y^{(n)}\big) \Big], \qquad \frac{\partial \text{obj}}{\partial \mathbf{w}} = \sum_{n=1}^{N} \frac{\partial \text{obj}}{\partial y^{(n)}} \frac{\partial y^{(n)}}{\partial a^{(n)}} \frac{\partial a^{(n)}}{\partial \mathbf{w}}.$$

Worked factor by factor, the objective contributes $\partial \text{obj}/\partial y^{(n)} = (y^{(n)} - t^{(n)})/(y^{(n)}(1 - y^{(n)}))$, the unit contributes $\partial y^{(n)}/\partial a^{(n)} = y^{(n)}(1 - y^{(n)})$, and the pre-activation contributes $\partial a^{(n)}/\partial \mathbf{w} = \mathbf{x}^{(n)}$. The awkward denominator is exactly the sigmoid derivative, so the two cancel, as the board shows by striking them out, and each term of the sum collapses to

$$\big(y^{(n)} - t^{(n)}\big)\mathbf{x}^{(n)}.$$

Residual times input. The cancellation is the point: it is why this pairing of output unit and loss is the one everybody uses, and it is the atom that backpropagation chains.

Backpropagation has a messy provenance. The slides are unusually careful here, and they begin further back than most treatments: the perceptron in 1957 [57], logistic regression in 1958 [11], and the XOR counterexample in 1969 [47]. Backpropagation was then popularised by Rumelhart, Hinton and Williams in 1986 [58], but Werbos had it in 1974 [70], and the slide states flatly that it "has been reinvented several times before 1986", pointing at Schmidhuber's history page [61]. What actually changed the field was not the rule but the tooling, and Chandar gives the timeline: Torch (2002), Theano (2007 to 2008), Caffe (2013), TensorFlow (2015), PyTorch (2016) and JAX (2018). Automatic differentiation, not the chain rule, is the enabling technology.

Expressivity is settled. The universal approximation theorem is quoted as Hornik, 1991 [32]: a single hidden layer network with a linear output unit can approximate any continuous function arbitrarily well, *given enough hidden units*. On the slide the final clause is highlighted and braced, and underneath, in red, sits the phrase that reframes everything after it: **expressivity versus learnability**. One hidden layer suffices in principle, which is precisely why the rest of the lecture is about optimisation rather than architecture.

Learnability is not. Depth turns the chain rule into a product of Jacobians,

$$\frac{\partial E}{\partial \mathbf{W}^{(T)}} = \frac{\partial E}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{h}_T} \underbrace{\left[\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_{T-1}} \frac{\partial \mathbf{h}_{T-1}}{\partial \mathbf{h}_{T-2}} \cdots \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \right]}_{\text{vanishing gradient problem}},$$

and the bracket is boxed on the board. The historical answer was to avoid training the depth all at once: greedy layerwise pretraining [2, 31], in which each layer is fitted as an autoencoder before the stack is finetuned end to end (Figure 2.1). The modern answer is quieter and lives in the activation function, where the lecture records $\text{Swish}(x) = x\,\sigma(x)$ [56] and $\text{SwiGLU}(x) = \text{Swish}(x\mathbf{W}) \otimes x\mathbf{V}$ [63].

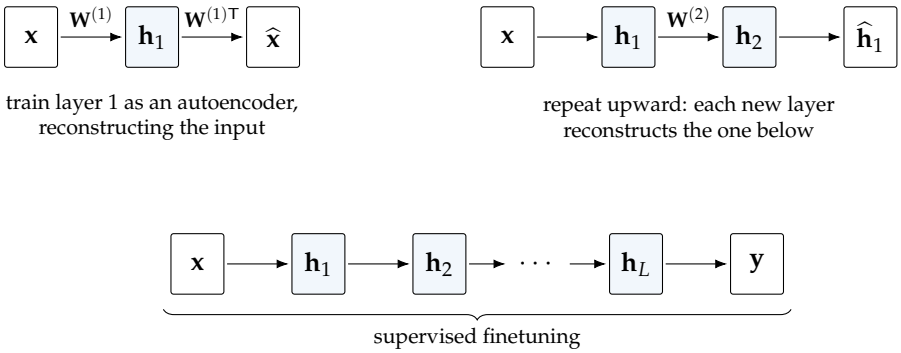

Figure 2.1: Greedy layerwise pretraining as drawn in the Introduction to Deep Learning lecture. Each layer is first trained to reconstruct the representation below it, so no gradient ever has to travel the full depth, and only then is the assembled stack finetuned against labels. Redrawn from photographs of the board.

Initialisation decides trainability. The lottery ticket hypothesis [20] is quoted verbatim: dense, randomly initialised feed-forward networks contain subnetworks, "winning tickets", that when trained in isolation reach test accuracy comparable to the original network in a similar number of iterations. Placed here, immediately after the vanishing-gradient material, it reads as a statement about optimisation rather than about compression: most of the network is scaffolding for finding a subnetwork the gradient can actually move.

The i.i.d. assumption is the frontier. The penultimate slide is four lines. Current focus: i.i.d. learning. True learning: non-i.i.d., namely sequential, online and continual. Nothing follows it except a pointer to Mark Schmidt's optimisation tutorial at ICML 2026. The lecture ends by naming its own limitation.

Notation

Superscript (n) indexes training examples, so $t^{(n)}$ is the target and $y^{(n)}$ the prediction for example n. Layer index is a superscript in parentheses on weight matrices, $\mathbf{W}^{(\ell)}$, and a subscript on activations, $\mathbf{h}_\ell$. This convention holds for later chapters, with one difference worth flagging now: Chandar writes the objective as an average over N examples with an explicit minus sign, whereas Cho (Chapter 6) works with expectations under a population distribution. The quantities are the same; the framing differs.

Why it matters

This chapter is the hinge for three later ones. The non-i.i.d. closing slide is picked up directly by Lyle on continual learning (Chapter 7), who turns "sequential, online, continual" into two concrete failure modes, and by Pascanu (Chapter 8), who argues that the i.i.d. assumption is what makes gradient-based credit assignment work at all. The lottery ticket hypothesis reappears in Alistarh's history of compression (Chapter 11), filed under pruning rather than optimisation, which is a real difference of emphasis between two speakers citing one paper.

> **NOT FROM THE LECTURE**
>
> The two readings of lottery tickets pull apart. Chandar places the result among answers to "how do we train deep networks", making it a claim about which initialisations admit a trainable path. Alistarh places it in the "Age of Discovery" of model compression, making it a claim about how much of a trained network is redundant. The paper supports both, but they suggest different experiments.

3 Representational Geometry and Alignment in Neural Nets

Phillip Isola • *MIT*

Isola's argument is that a representation should be identified with the distances it induces, not with the coordinates it happens to use, and that almost everything interesting follows from taking that seriously. If two networks encode the same distances in different bases they are the same representation for practical purposes, so the right way to compare them is to compare kernels. Once comparison is on that footing, convergence becomes measurable: strong models trained on different data and different modalities are converging in kernel structure, and human perceptual judgements sit close to it. The lecture ends by turning this from an observation into an engineering target, and states what that costs.

Key ideas

Every layer is a representation, and a representation is a point cloud. A representation is a map $\phi : \mathcal{X} \to \mathcal{Z}$, usually with $\mathcal{Z} = \mathbb{R}^d$, and the representation of a dataset is the set $Z = \{\phi(\mathbf{x}^{(i)})\}_{i=1}^n$, a point cloud in $\mathbb{R}^d$. Deep nets transform that cloud layer by layer, so the input data and the output beliefs are representations too, not just the penultimate activations. The framing pays off immediately in what a classifier is doing: deforming an input point cloud until the classes land on the vertices of a simplex. The opening example is older, that object detectors emerge in scene CNNs trained only on scene labels [75], which establishes that internal activations carry semantics nobody put there on purpose.

Comparing two clouds means choosing an invariance. Representational similarity is a function $d : \phi_1 \times \phi_2 \to \mathbb{R}$, in practice evaluated on finite samples Z_1 and Z_2. Isola's organising observation is that every metric measures sameness only up to some transformation T, so choosing the metric *is* choosing which differences are being declared irrelevant. Regression-based metrics, inherited from neuroscience, fit a map from one embedding to the other,

$$d(Z_1, Z_2) = \frac{1}{n}\sum_{i=1}^{n} \big\| h^*(z_1^{(i)}) - z_2^{(i)} \big\|, \qquad h^* = \arg\min_h \frac{1}{n}\sum_{i=1}^{n} \big\| h(z_1^{(i)}) - z_2^{(i)} \big\|,$$

so they quotient out whatever the fitted map h is allowed to be. Kernel-alignment metrics instead characterise a representation by its own internal geometry, $K(\mathbf{x}_i, \mathbf{x}_j) = \big\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \big\rangle$, and compare those. Because rigid transformations leave distances untouched, kernel alignment is automatically invariant to rotation, translation and reflection, and it is indifferent to embedding dimension, so a 512-dimensional and a 4096-dimensional model can be compared directly. The deck's worked example of a cross-modal embedding is CLIP [55].

CKA, and why Isola prefers this family. Centered kernel alignment adds invariance to isotropic scaling on top of the isometries [39]:

$$\mathrm{CKA}(K_1, K_2) = \frac{\mathrm{tr}(K_1 \mathbf{H} K_2 \mathbf{H})}{\sqrt{\mathrm{tr}(K_1 \mathbf{H} K_1 \mathbf{H})\ \mathrm{tr}(K_2 \mathbf{H} K_2 \mathbf{H})}},$$

where $\mathbf{H}$ is the centring matrix, so the numerator is kernel similarity with the mean similarity subtracted and the denominator normalises for scale. A cheaper relative asks what fraction of a point's nearest neighbours under ϕ_1 remain its nearest neighbours under ϕ_2 [33]. Asked which family is best, Isola gives an opinion rather than a theorem: kernel alignment, because distance is what downstream tasks consume. Two representations related by a linear isometry are interchangeable for retrieval, k-nearest-neighbour classification, minimum-norm linear regression and MLPs in the NTK regime. He goes further and suggests making it definitional, treating a representation as nothing but a specification of $d : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$.

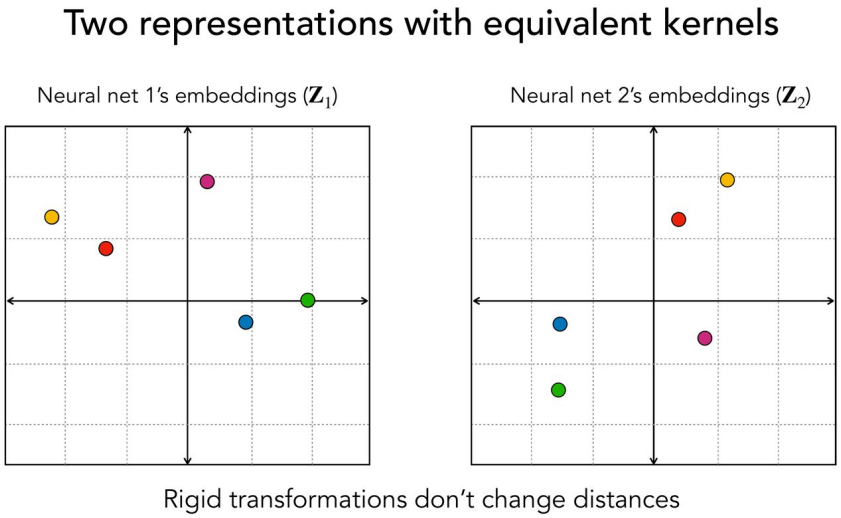

Figure 3.1: The invariance Isola argues for, from the Representational Geometry lecture. The two panels hold the same five points related by a rigid transformation: every coordinate changes, no pairwise distance does. The kernels are therefore identical and the two networks count as the same representation, whereas a coordinate-wise comparison sees two unrelated clouds.

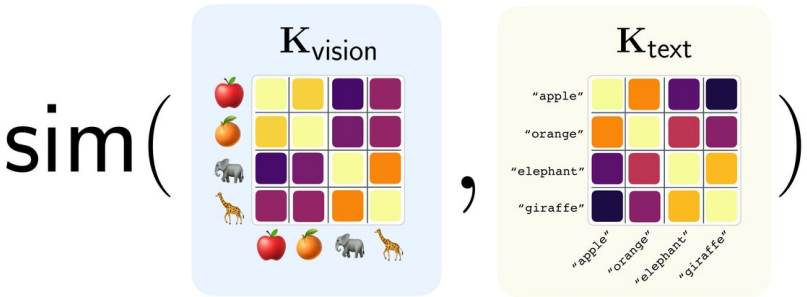

Figure 3.2: What cross-modal comparison actually compares, from the Representational Geometry lecture. Each model is reduced to its own kernel matrix over the same four concepts, one from images and one from the words naming them, and the two matrices are then compared. Nothing requires the embeddings to share a dimension or a basis, only that both induce distances over the same items.

Convergence, measured against humans and across modalities. With a metric fixed, the empirical section asks which representations actually agree. Against human similarity judgements collected in DreamSim [23], DINO [7] came out closest, which is notable

because DINO is trained on no human judgements at all. The same machinery, borrowed from representational similarity analysis in neuroscience [40, 71], then compares a vision model to a language model by asking whether their kernels are alike (Figure 3.2). The reported answer is that stronger models converge, and that alignment increases with scale and over time [33], which is the trend in Figure 3.3.[1] The lecture refuses to leave "same" vague and offers three hypotheses: convergence to the same embedding vectors (identity), to the same global kernel (glocal isometry), or to the same local neighbourhoods and clusters (local ordinal isometry). Two theoretical results are sketched. A contrastive learner with an NCE objective converges to pointwise mutual information,

$$\langle \phi(\mathbf{x}_a), \phi(\mathbf{x}_b) \rangle \;\approx\; \log \frac{P_{\text{co}}(\mathbf{x}_a, \mathbf{x}_b)}{P_{\text{co}}(\mathbf{x}_a)\, P_{\text{co}}(\mathbf{x}_b)},$$

so for bijective discrete observation functions the PMI over observations equals the PMI over underlying events, which forces different modalities onto the same kernel. Separately, two CLIP-like models are claimed to converge to representations related by an orthogonal transform $\mathbf{Q}$ [28].

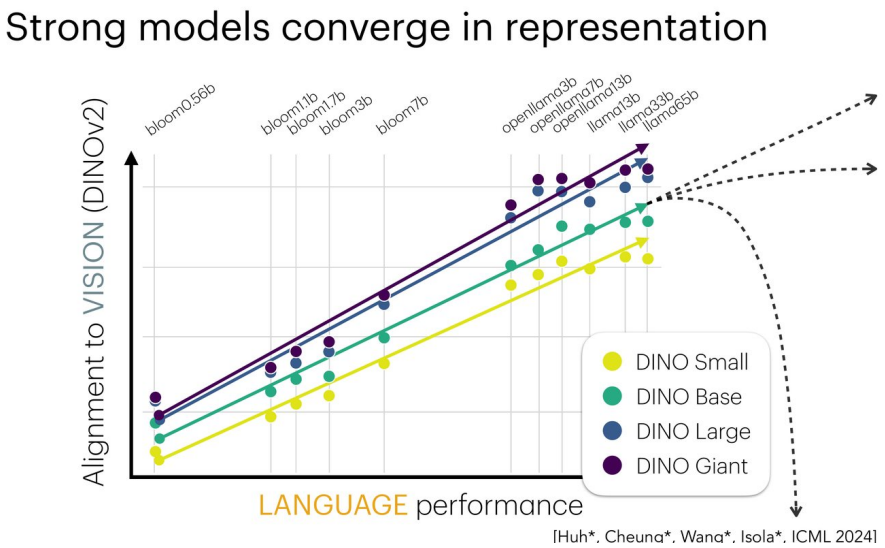

Figure 3.3: The central empirical claim of the Representational Geometry lecture. Alignment of a language model's kernel to a vision model's (DINOv2) rises with the language model's own performance, across the BLOOM and LLaMA families and across four DINO sizes. Better language models look more like vision models, without ever having seen an image.

Alignment as a target, and what it costs. If models converge anyway, alignment can be engineered. Aligned representations let models share supervision and data, and a common representation is a bridge for translation. The striking part is that paired data is not required: vec2vec translates between embedding spaces using a GAN with a cycle-consistency loss and a kernel-matching loss [35], an idea with a direct ancestor in unsupervised word translation [10], while REPA aligns a generative model's representation to a pretrained one [72]. Isola then lists the detriments without softening them: alignment reduces diversity in the population of models; where one modality has access to qualitatively different information, alignment destroys exactly that information; and

[1] The personal notes add a caveat absent from the slides, that open-source LLMs "hasn't really increased though". Recorded as a contemporaneous observation, not as a lecture claim.

there may be no single best representation for all problems, which he notes is not merely a practical worry but true in theory.

Notation

ϕ and ψ denote representations, $z = \phi(\mathbf{x})$ an embedding, Z a point cloud over a dataset, h a map fitted between two embeddings, K a kernel matrix and $\mathbf{H}$ the centring matrix.

Why it matters

This is the lecture the personal notes cover in most detail, and it connects outward more than any other. The claim that distance is the only thing to preserve is the same instinct that drives geometric deep learning (Chapter 12), where invariance is imposed by construction rather than measured after the fact. The convergence story is a counterpoint to Cho on estimation (Chapter 6): Isola measures agreement between two learned objects, Cho asks whether a learned estimator is even unbiased. And the tension between alignment and diversity bears directly on CLIP's contrastive objective, which the summer school's multimodal tutorial builds from scratch, since that loss is precisely the mechanism the theory here predicts will drive convergence.

> **NOT FROM THE LECTURE**
>
> Isola's stronger claim, recorded in the personal notes rather than on the slides, is that there is "no future for distance functions that are not learned". That sits in tension with the argument of the lecture, which uses fixed metrics such as CKA to establish convergence. If the yardstick is itself fitted to the objects being measured, the finding that models converge becomes harder to state without circularity.

4 Diffusion Models

Sander Dieleman • *Google DeepMind*

Diffusion is one of two ways to break generation into easy steps, and almost every design decision people argue about turns out to be downstream of a single equation for the forward corruption process. Once $\mathbf{x}_t = \alpha(t)\mathbf{x}_0 + \sigma(t)\boldsymbol{\varepsilon}$ is on the board, the variance-preserving, variance-exploding and flow-matching families are three choices of α; predicting $\mathbf{x}_0$, $\boldsymbol{\varepsilon}$ or a velocity are interconvertible parameterisations of one object; guidance is Bayes' rule applied to the score; and the question of why diffusion works so well on images has a spectral answer that Dieleman states as a slogan. The final third is about where the framework strains: distillation, discrete data, and what a latent space is for.

Key ideas

Two roads to iterative refinement, and the space matters more than the road. Iterative refinement breaks the generative procedure into simpler steps in which all steps are of comparable difficulty. Autoregression does this with the chain rule of probability, $p(\mathbf{x}) = \prod_i p(\mathbf{x}_i \mid \mathbf{x}_{<i})$, and Dieleman's historical sequence makes the point that the interesting variation was never the factorisation but the space it acts on: pixels in PixelRNN and PixelCNN [51], raw amplitudes in WaveNet [53], discrete latents in VQ-VAE [52]. Diffusion factorises differently, by iterative denoising, and the same lesson applies: production systems run diffusion in a latent space, not in pixels, compressing a $3 \times 256 \times 256$ image to something like $8 \times 32 \times 32$ before the diffusion model ever sees it.[1]

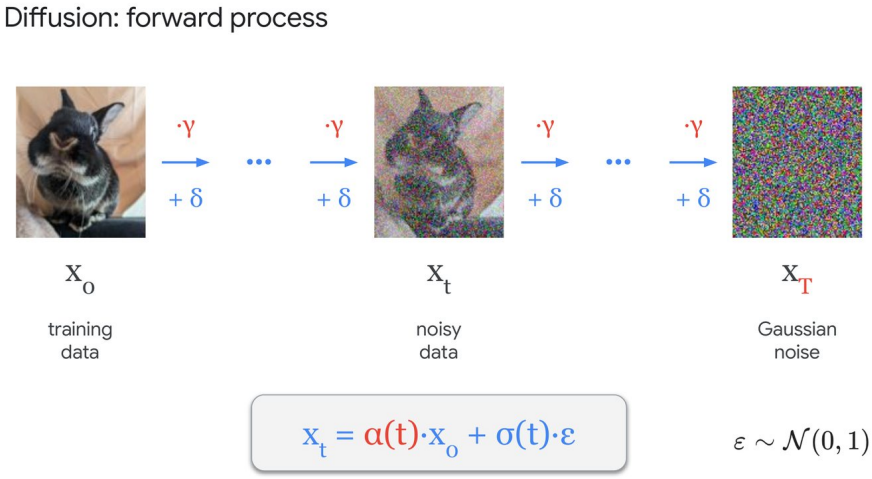

Figure 4.1: The forward process, from the Diffusion Models lecture. Each step scales by γ and adds noise δ, carrying training data $\mathbf{x}_0$ through noisy data $\mathbf{x}_t$ to pure Gaussian noise $\mathbf{x}_T$. The whole chain collapses to the boxed line, $\mathbf{x}_t = \alpha(t)\mathbf{x}_0 + \sigma(t)\boldsymbol{\varepsilon}$: corruption is just an interpolation between the data and a sample of noise.

[1] This is the point the personal notes flag as "Diffusion is done in latent space, not in pixel space!". The deck supports it with a pixels-versus-latents slide and a pointer to regularising the latent space for generative modelling.

One equation for the forward process. Corruption interpolates between data and noise (Figure 4.1),

$$\mathbf{x}_t = \alpha(t)\,\mathbf{x}_0 + \sigma(t)\,\boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}),$$

where $\sigma(t)$ is the noise schedule controlling the rate of corruption. The families differ only in α: variance-preserving takes $\alpha = \sqrt{1-\sigma^2}$, variance-exploding takes $\alpha = 1$, and rectified flow or flow matching takes $\alpha = 1 - \sigma$. Presented this way, a literature that usually reads as three separate methods collapses into one line with a parameter.

What the network predicts is a free choice. Since $\mathbf{x}_t$ is a linear combination of $\mathbf{x}_0$ and $\boldsymbol{\varepsilon}$, a prediction of one converts into a prediction of the other, so $\mathbf{x}_0$-prediction, $\boldsymbol{\varepsilon}$-prediction, v-prediction with $v = \alpha(t)\boldsymbol{\varepsilon} - \sigma(t)\mathbf{x}_0$ [59], and the flow-matching target $\boldsymbol{\varepsilon} - \mathbf{x}_0$ [45] are reparameterisations rather than different models. The same object appears again as the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$, which points toward regions of high density, so knowing the gradient is knowing p [34, 68]. In practice all of these are trained by minimising a mean squared error.

Guidance is Bayes' rule, twice. Dieleman's own phrase, and the one the personal notes record, is that guidance is "a cheat code": it buys sample quality at the cost of diversity. Classifier guidance is derived on the slide straight from Bayes' rule, in three lines:

$$\begin{aligned} p(\mathbf{x} \mid c) &\propto p(c \mid \mathbf{x})\, p(\mathbf{x}), \\ \log p(\mathbf{x} \mid c) &= \log p(c \mid \mathbf{x}) + \log p(\mathbf{x}) + C, \\ \underbrace{\nabla_{\mathbf{x}} \log p(\mathbf{x} \mid c)}_{\text{conditional score}} &= \underbrace{\nabla_{\mathbf{x}} \log p(c \mid \mathbf{x})}_{\text{classifier gradient}} + \underbrace{\nabla_{\mathbf{x}} \log p(\mathbf{x})}_{\text{unconditional score}} . \end{aligned}$$

The normalising constant differentiates away, which is the whole trick: the conditional score is the unconditional score plus a classifier gradient, so an unconditional model becomes conditional without retraining. A scale factor w on the second term acts as an inverse temperature, and the lecture is careful about what it sharpens: it sharpens $p(c \mid \mathbf{x})$, which is not the same thing as sharpening $p(\mathbf{x})$ or $p(\mathbf{x} \mid c)$. Classifier-free guidance drops the separate classifier: predict $\widehat{\mathbf{x}}_0$ and $\widehat{\mathbf{x}}_{0|c}$ from one model trained with conditioning dropout, take the difference δ, and amplify it by w. The insight is that δ is itself a Bayesian classifier, so this is Bayes' rule applied twice, and it is less prone to adversarial examples than guiding with a real classifier [50]. Figure 4.2 shows guidance as a geometric operation on one sampling step, not a change to the model.

Diffusion is approximate spectral autoregression. The lecture's answer to why diffusion works so well on images is spectral, and it is the insight the personal notes single out. Natural images have a power spectrum that falls off steeply with spatial frequency, while Gaussian noise is flat. Adding noise at level σ therefore does not damage all scales equally: it drowns everything above the frequency where the flat noise floor crosses the image's decaying spectrum, and leaves the low frequencies almost intact (Figure 4.3). Sampling, run backwards, is then recovering frequency bands roughly coarsest-first, which is why Dieleman puts it as diffusion being approximate spectral autoregression [16]. It also explains why the trick transfers poorly to data without a decaying spectrum.

Where the framework strains. Three limitations close the lecture. Distillation into fewer sampling steps requires cutting corners, and the corners are entropy and diversity first, then visual fidelity [17, 64]. Discrete data breaks the score outright, since $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ is undefined when $\mathbf{x}$ is discrete; the two escapes are to define a discrete corruption process,

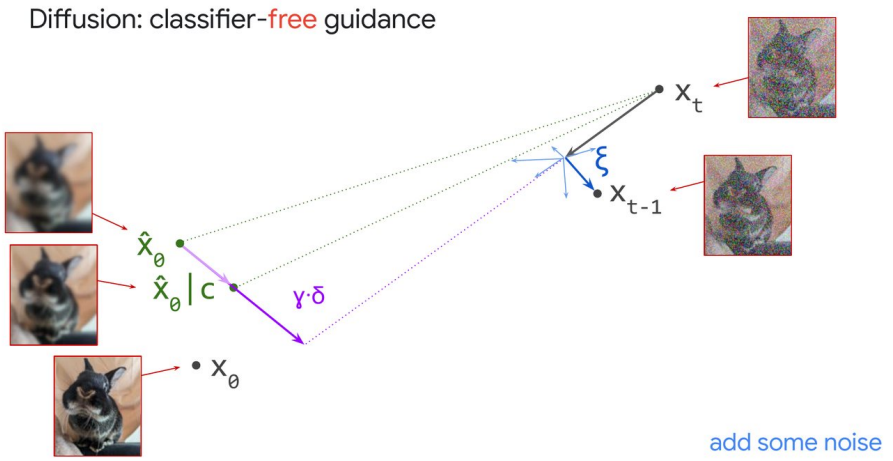

Figure 4.2: Classifier-free guidance as one sampling step, from the Diffusion Models lecture. From the noisy state $\mathbf{x}_t$ the model predicts both an unconditional $\widehat{\mathbf{x}}_0$ and a conditional $\widehat{\mathbf{x}}_{0|c}$; their difference δ is scaled by γ and extrapolated past the conditional prediction, then a small step is taken toward that extrapolated target and fresh noise ξ is added to reach $\mathbf{x}_{t-1}$. Note that the guided target overshoots $\mathbf{x}_0$ entirely, which is exactly why guidance trades diversity for apparent quality.

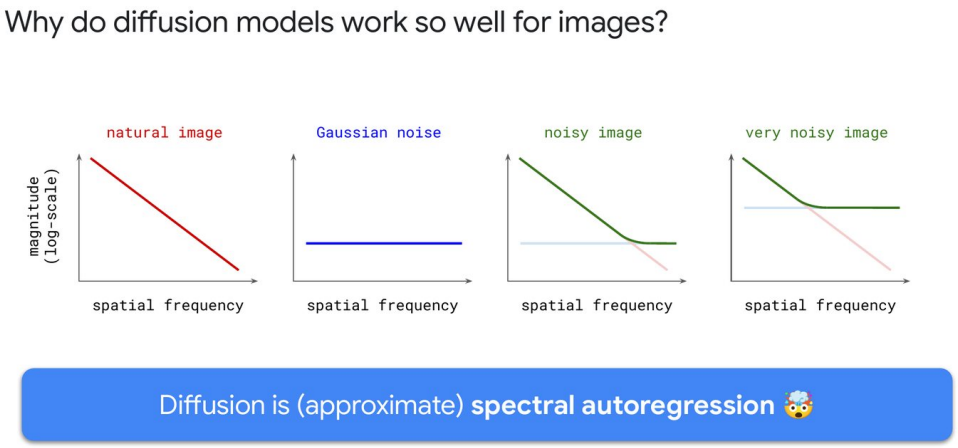

Figure 4.3: The spectral argument, from the Diffusion Models lecture. A natural image's magnitude spectrum falls steeply with spatial frequency while Gaussian noise is flat, so their sum is the pointwise maximum: at moderate noise the high frequencies are already replaced by the noise floor, and at high noise only the lowest frequencies survive. Read right to left, sampling restores frequency bands coarsest-first, which is the sense in which diffusion is approximate spectral autoregression.

or to embed the data in a continuous space, which is what CDCD does while deliberately making diffusion look familiar to language-model practitioners by using an unmasked Transformer and a cross-entropy loss instead of an MSE [19]. And data curation is named as essential for high-quality results while being structurally disincentivised in the research community.

Notation

$\mathbf{x}_0$ is clean data, $\mathbf{x}_t$ the corrupted state at time t, $\boldsymbol{\varepsilon}$ standard Gaussian noise, $\alpha(t)$ and $\sigma(t)$ the interpolation coefficients, and w the guidance scale. The deck writes the guidance scale as a Greek gamma. It is w here, to keep it clear of the discount factor in Chapter 5.

Why it matters

The parameterisation argument travels furthest. Whenever a literature presents n competing methods, ask whether they are n choices of one coefficient. The spectral analysis connects directly to Stanković's lecture (Chapter 10), which also treats a spectrum as the object to manipulate, though on graphs rather than grids, and to Veličković's argument (Chapter 12) that the structure of the data domain should dictate the model. Classifier-free guidance reappears in the summer school's multimodal tutorial, where it is the single line the notebook asks to be written by hand.

NOT FROM THE LECTURE

Two threads to pull. The personal notes record the claim that diffusion models are essentially RNNs, which permits training much deeper networks. That framing is not on the slides in those words, though it is consistent with the iterative-refinement picture where one network is applied repeatedly with a time input; treat it as a remark from the room rather than a slide. Second, the deck's closing list of untouched topics includes flow maps, also flagged in the personal notes, pointing at Dieleman's blog [18]. That is the natural continuation of the unification of diffusion and flow matching set out above.

5 Introduction to Reinforcement Learning

Andreea Deac • *Isomorphic Labs*

The lecture is built on one identity used three ways. Bellman's equation, $V^\pi(s) = \mathbb{E}[r_t + \gamma V^\pi(s')]$, admits a full-expectation backup, a one-step bootstrapped backup and a full-depth sampled backup, and dynamic programming, temporal difference learning and Monte Carlo are exactly those three choices. Everything else in the lecture is either what goes wrong in the absence of that identity, namely REINFORCE's inability to assign credit to individual actions, or what has to be bolted on afterwards to make policy gradients survive contact with their own data. The last section lands on GRPO and RLVR, and the framing is that the modern LLM stack is this material with a verifiable reward substituted for a learned one. Deac also ran the talk as a live bandit, and reported that it failed.

Key ideas

The problem is defined by where the data comes from. Four contrasts with supervised learning organise the setup: the data is self-induced by acting rather than a fixed i.i.d. set of pairs, the signal is a reward rather than a correct label, feedback may arrive many steps later rather than immediately, and the goal is to act so as to maximise long-term return rather than to learn a map. Formally an MDP is $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with $P(s' \mid s, a)$ and $R(s, a) = r$, the Markov property meaning the future depends on the present state alone. A policy $\pi(a \mid s)$ produces a trajectory $\tau = (s_0, a_0, r_0, s_1, \dots)$ scored by the return $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$, and the objective is $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$. The deck's own reference slide points at Sutton and Barto for all of this [65].

The motivation given for γ is arithmetic rather than assertion. Three gridworld trajectories score 99, 100 and 101 undiscounted, so the greedy detour wins and nothing prevents the agent cycling through a $+100$ cell forever. At $\gamma = 0.95$ the same three trajectories, of length 3, 7 and 9 steps, score 89.2, 73.5 and 67.1: the short path now wins, and the infinite loop sums to about 10 rather than diverging.

Policy gradients, and the credit they cannot assign. The direct approach is gradient ascent on J, but θ sits in the sampling distribution rather than in the return, so differentiating through G after sampling gives zero and the trajectories cannot be enumerated. The log-derivative trick moves the gradient inside the expectation,

$$\nabla_\theta \, \mathbb{E}[f] = \sum_\tau f \nabla_\theta p = \sum_\tau f \frac{\nabla_\theta p}{p} p = \mathbb{E}\left[f \nabla_\theta \log p\right],$$

which yields the policy gradient theorem, $\nabla_\theta J = \mathbb{E}_\tau\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t\right]$: push up the log-probability of actions in proportion to how good the outcome was, and never differentiate through the environment. The lecture then immediately shows the cost of that convenience (Figure 5.1). Credit attaches to whole action sequences, so 58 of 104 genuinely bad actions were pushed up simply because the episode they belonged to went well.

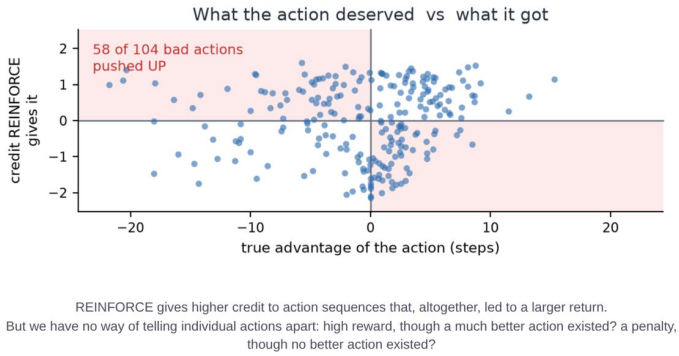

Figure 5.1: Why REINFORCE needs a baseline, from the Introduction to Reinforcement Learning lecture. Each point is one action, with its true advantage on the horizontal axis and the credit REINFORCE assigned on the vertical. The shaded quadrants are the mistakes: actions that hurt but were reinforced, and actions that helped but were penalised. The lecture's count is 58 of 104 bad actions pushed up.

One Bellman equation, three backups. Value functions supply the missing notion of what to expect. Writing Bellman's equation out,

$$V^\pi(s) = \mathbb{E}\big[r + \gamma V^\pi(s')\big] = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a)\big[r + \gamma V^\pi(s')\big],$$

the recursion $G = r + \gamma G'$ is what lets the problem be solved backwards, illustrated with a Montenegrin road map: if the best route from Podgorica to Kotor passes through Cetinje, the best route from Cetinje to Kotor is part of it. The three ways to use it (Figure 5.2) are dynamic programming, taking the full expectation and needing the model P; temporal difference, $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$, one sampled step bootstrapped off its own estimate; and Monte Carlo, $V(s) \leftarrow V(s) + \alpha[G_t - V(s)]$, one sample to termination. Monte Carlo is unbiased with high variance; TD reverses the trade, taking bias from the Markov assumption and from trusting its own current estimates. An eight-episode exercise makes the difference concrete: with $V(B) = 6/8$, batch Monte Carlo puts $V(A) = 0$ because the one episode through A returned 0, while batch TD puts $V(A) = 0.75$ because the transitions imply A leads to B with certainty. Monte Carlo fits the observed data, TD fits the MDP that the data implies.

Making policy gradients survive their own data. Subtracting a baseline $b(s)$ leaves the gradient unbiased, since $\sum_a b(s)\nabla_\theta \pi_\theta(a \mid s) = b(s)\nabla_\theta 1 = 0$, and reduces variance; using V as the baseline gives the advantage $A(s,a) = Q(s,a) - V(s)$, which also puts states with different reward scales on a common footing. Actor-critic learns both, and generalised advantage estimation reintroduces the n-step bias-variance dial, $A_t^{\mathrm{GAE}} = \sum_{k\geq 0}(\gamma\lambda)^k[r_{t+k} + \gamma V(s_{t+k+1}) - V(s_{t+k})]$, with $\lambda = 0$ high bias and $\lambda = 1$ low bias. Two failure modes follow. Premature collapse is met with entropy regularisation, $\theta \leftarrow \theta + \eta\nabla_\theta(J(\theta) + \beta H(\pi_\theta))$, annealing β down. Instability is worse: the data comes from the current policy, so one oversized update breaks behaviour and the next batch is collected by the broken policy, a negative spiral. TRPO constrains it, $\max_\theta \mathbb{E}[(\pi_\theta/\pi_{\mathrm{old}})A]$ subject to $D_{\mathrm{KL}}(\pi_{\mathrm{old}} \,\|\, \pi_\theta) \leq \delta$, at the price of a second-order solve; PPO clips the ratio instead for the same effect with first-order optimisation.

Where this lands in 2026. The reward changes character: RLHF's learned model of human preference gives way to RLVR's verifiable checker, does the test pass, is the answer

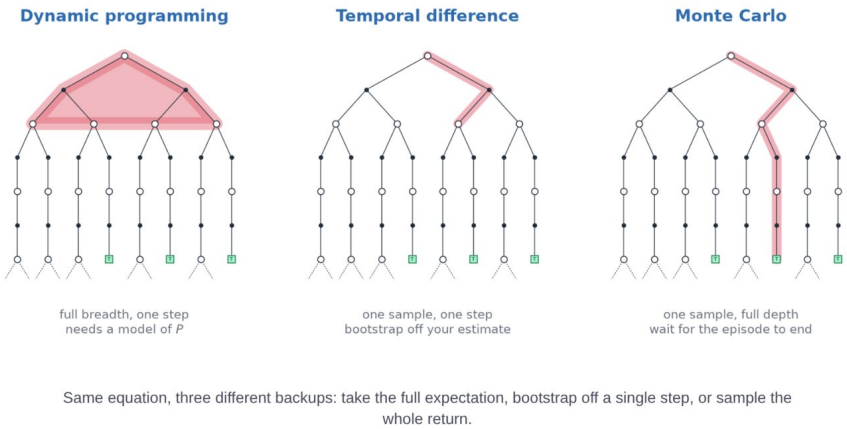

Figure 5.2: The organising figure of the Introduction to Reinforcement Learning lecture, titled "Three ways to use Bellman". Shading marks which part of the tree each backup consults: dynamic programming sweeps one step in full breadth, temporal difference follows a single sampled step and then bootstraps, Monte Carlo follows one sampled path to a terminal state. The equation is the same in all three cases.

right, which is cheaper, harder to hack and scalable [43]. DeepSeek-R1 [15] is presented as evidence that RL on a base model with no supervised finetuning can elicit reasoning and self-correction. GRPO [62] is called "the one algorithm to teach": PPO must carry a critic typically the size of the policy, whereas GRPO samples a group of answers per prompt and standardises within the group, $A_i = (r_i - \text{mean})/\text{std}$, with no critic network at all. Genie 3 [4] is contrasted with dynamic programming: where DP needs an explicit P, Genie learns implicit dynamics from data and generates interactive future states from text, so environments themselves become generatable.

Notation

$\mathcal{S}, \mathcal{A}, P, R, \gamma$ for the MDP, π for a policy, τ for a trajectory, G_t for the return, V and Q for state and action values, A for the advantage, α for the value-update step size and η for the policy step size.

Why it matters

This chapter is the prerequisite for the summer school's reinforcement learning tutorial, which implements REINFORCE, PPO and GRPO in that order and so retraces this argument in code. The i.i.d. discussion around experience replay is the same concern Chandar closed on (Chapter 2) and that Lyle and Pascanu develop (Chapters 7 and 8): replay exists because SGD assumes independent samples and consecutive transitions are not independent, which is continual learning's problem solved by brute force.

NOT FROM THE LECTURE

The most useful thing in this lecture is not on a slide about RL at all. Deac turned the talk into a nine-armed bandit over slide layouts and background colours, with audience votes as reward, and then reported the outcome honestly: 53 rounds, 0 votes, every round skipped. The closing slide lists why real-world RL is hard using her own failure as the example: reward confounded with content, non-stationarity as the audience tires, sparse and noisy feedback, varying turnout, delayed and batched rewards, and an assumption that the two dimensions

> were independent when they were not. Any applied RL project should expect to fail in at least two of those ways before it fails in an interesting one.

6 Statistics: Learning to Estimate

Kyunghyun Cho • *New York University*

Cho's lecture has one thesis, stated two-thirds of the way through, and it reorganises a great deal of statistics: inference does not have to be about inverting the generative process. The conventional route to any estimand is to model the data generating distribution and then invert it, and the lecture's claim is that this step is optional. Given the ability to sample problems with known answers, estimator design becomes a supervised regression problem, and gradient descent finds an estimator that beats the textbook one. The argument is built on a deliberately trivial example, the population standard deviation, then repeated for mutual information, Bayesian predictive distributions and causal effects, and finally turned back on theory to redefine what identifiability means.

Key ideas

Start with something embarrassingly simple. The sample standard deviation is a biased estimate of the population standard deviation, because the square root sits outside the expectation. Rather than deriving a correction, Cho asks why estimator design is not simply a minimisation problem. Given pairs $(\mathcal{D}, \theta)$ of a sample set and the true population standard deviation, a good estimator F has low MSE, and the only source of variation is the sampling that produced $\mathcal{D}$. So define a meta-distribution P_Γ over distributions of known population standard deviation and minimise over a hypothesis class $\mathcal{F}$ of estimators,

$$\min_{F \in \mathcal{F}} \mathop{\mathbb{E}}_{\pi \sim P_\Gamma} \Big[\mathop{\mathbb{E}}_{\mathcal{D} \sim \pi} \big[\mathrm{MSE}\big(F(\mathcal{D}), \mathrm{stdev}(\pi)\big) \big] \Big].$$

As the slides note, this is exactly training a regression model: a data distribution P_Γ, a set of samples as input, a real number as output, and an MSE loss.

The architecture is forced by an invariance. An estimator over a sample set must be permutation invariant, so F is a transformer with the positional embeddings removed, followed by aggregation, which the lecture calls Set Transformer++ [74].[1] This is a small but neat instance of the geometric deep learning argument (Chapter 12): the symmetry of the data dictates the architecture, here without either speaker mentioning the other.

It works, and the reason it works is the interesting part. The learned estimator is essentially unbiased where `np.std` is not, is more sample-efficient, converges as the sample size grows, and, importantly, works on log-normal distributions that were held out of training (Figure 6.1). Cho's reading of that last fact is the pivot of the lecture. Had the network been implicitly identifying the underlying distribution, fitting it, and then correcting the sample standard deviation, it would fail on a family it had never seen. It does not fail, so it is doing something else: bypassing density estimation and going straight to inference. Supervised learning found a new estimator of the population standard deviation.

[1] "Set Transformer++" is the lecture's own name for the architecture. The cited paper is on set norm and equivariant skip connections for deep sets, and does not use that name.

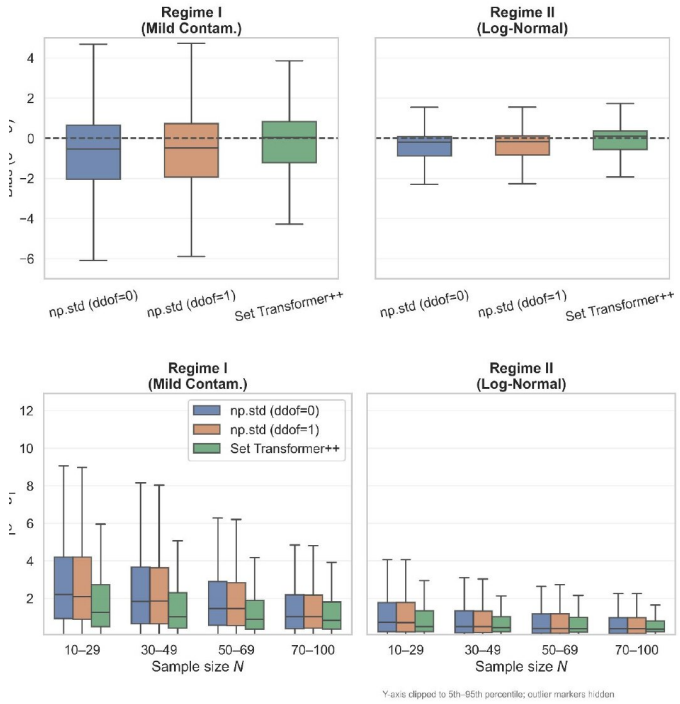

Figure 6.1: The proof of concept from the Statistics lecture. Top, bias $\widehat{\sigma} - \sigma$ for two `numpy` estimators and the learned one, under mild contamination and under a log-normal regime that was held out of training. Bottom, absolute error against sample size. The learned estimator is centred on zero where `np.std` sits below it, and it holds up on the unseen family, which is the evidence that it is not implicitly fitting a density.

The same move, three more times. Mutual information is defined through density ratios, so the reflex is to estimate the ratio; MIST instead attends over both the sample axis and the dimension axis and predicts MI directly, reportedly outperforming existing estimators, keeping both bias and variance low, largely escaping the curse of dimensionality, and generalising to dimensionalities and sample sizes outside its training range. Once available, a learned estimator can itself serve as an objective function. The Bayesian predictive distribution is defined through the posterior, so the reflex is to approximate the posterior; prior-data fitted networks approximate the predictive distribution directly with a transformer [49], matching exact Gaussian-process predictions closely at a fraction of the cost of MCMC. That raises the obvious objection, where did the prior go, and the answer is to supply it in context: sample a prior, sample datasets from it, and train multi-task in-context learning to condition on both. Causal effects get the same treatment under the name black-box causal inference [6], recovering estimators for observed confounders and for instrument variables, and tested on the LaLonde job-training data [42]. A recurring caveat is that diversity of the training prior, not just its correctness, determines how far the estimator generalises.

Computational identifiability. The theoretical payoff is a redefinition. Mathematical identifiability says that if two observations of infinitely many samples have matching distributions then the parameters of interest must coincide. Cho proposes *computational* identifiability instead [5], defined relative to three operational constraints: a prior over the problem space, a hypothesis space, and an optimisation algorithm. The procedure is explicit. Take a causal query; define a prior over structural causal models combining given and simulated distributions; define a hypothesis space reachable by the chosen optimiser;

search for the best estimator; and if it is accurate for most of the sampled SCMs, declare the query identifiable. Unlike the mathematical version this is soft, so $p(\text{identifiable} \mid \varepsilon)$ becomes a quantity that can be computed and plotted as a curve.

Notation

θ is the estimand, $\widehat{\theta}$ or $F(\mathcal{D})$ the estimate, $\mathcal{D}$ a sample set, π a single data distribution and P_Γ the meta-distribution over such distributions. The distinction between π as a distribution over data here and π as a policy in Chapter 5 is the speakers' own; both are standard in their fields and neither is changed.

Why it matters

This lecture cuts against Chapter 3. Isola measures whether learned representations agree, using fixed metrics as the yardstick; Cho's programme suggests the yardstick itself should be learned, which is the same instinct as Isola's own remark that there is no future for unlearned distance functions. Read together they point at a world where both the model and the measurement are fitted, and where the burden shifts entirely onto the prior over problems. The bias-variance material also connects to Alistarh's linearity theorem (Chapter 11), which is another attempt to predict an error rather than measure it.

NOT FROM THE LECTURE

The assumption underneath all of this is easy to miss because the examples are so clean. Every learned estimator here is only as good as the meta-distribution P_Γ it was trained over, and for the standard-deviation example we can write that prior down honestly. For causal queries we cannot, which is precisely why computational identifiability is defined relative to a prior rather than absolutely. That makes it an honest notion, but it also means a query can be declared identifiable under one prior and not another, and the lecture does not claim otherwise.

7 Continual Learning

Clare Lyle • *Google DeepMind*

Lyle's framing is that continual learning is two problems, not one, and that standard training is bad at both. A network must remember what it learned before, and it must remain able to learn something new. The first failure is catastrophic forgetting and has been known since the early 1990s; the second is loss of plasticity and is the subject of most of the lecture, because it is both less obvious and better understood mechanistically. The line that organises everything is slide 17, which reads: "Continual learning doesn't have to be hard (unless you want to use a neural network)". The difficulties are properties of the optimiser and the architecture, not of the problem statement, and once diagnosed that way most of them have mundane fixes.

Key ideas

Two problems and a meta-problem. Problem one is remembering the past [38]. Problem two is retaining plasticity, the capacity to keep learning at all. The meta-problem is evaluation, and neither option is satisfactory: train on a pre-specified sequence of tasks, which is artificial, or find an application that is naturally continual, which is uncontrolled. That tension sits under every result in the field.

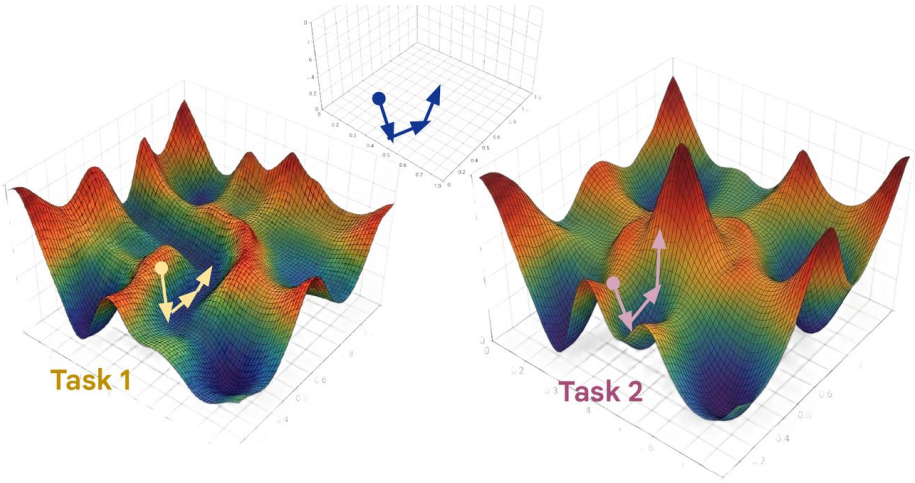

Figure 7.1: The geometric intuition for catastrophic forgetting, from the Continual Learning lecture. The same parameters sit on two different loss surfaces, one per task, and the descent directions the two tasks ask for are not aligned, as the inset axes make explicit. A step that lowers the task-two loss can climb the task-one loss, so progress on the new task is paid for out of the old one.

Forgetting is gradient conflict, and the fixes follow from that. The intuition given is geometric (Figure 7.1): gradients on different tasks often conflict, so a step that helps task two moves parameters out of the region that solved task one. Three families of solution follow directly from the diagnosis, and the lecture states them in plain terms before naming any method. Do not change the parameters used for the old task too much, which is regularisation and, with a different mechanism, sparsity and network expansion.

Keep the old data around and keep training on it, which is replay. Or periodically reset the model and retrain on everything, which pairs with distillation. Lyle's practical verdict appears again in the summary: if the data can be kept, the first problem is easy. Only under a memory constraint does it require finesse.

Loss of plasticity has a mechanism, and it is the optimiser. Under repeated task changes networks become less trainable over time, in the extreme case with dead ReLUs and a model emitting a uniform distribution. Two causes are separated. First, gradual convergence toward bad parameters, diagnosed through the empirical NTK matrix at convergence. Second, and more surprising, destructive updates that happen almost immediately after a task change. The reason is that optimisers are built for stationary problems. In Adam the second moment estimate decays slowly, with the usual 0.99, so it reflects the small gradients seen at the end of the previous task; when the loss changes the numerator grows at once while the denominator lags, and the effective step size becomes enormous (Figure 7.2). The loss landscape has meanwhile evolved to be stable only at the small step size it was trained with.

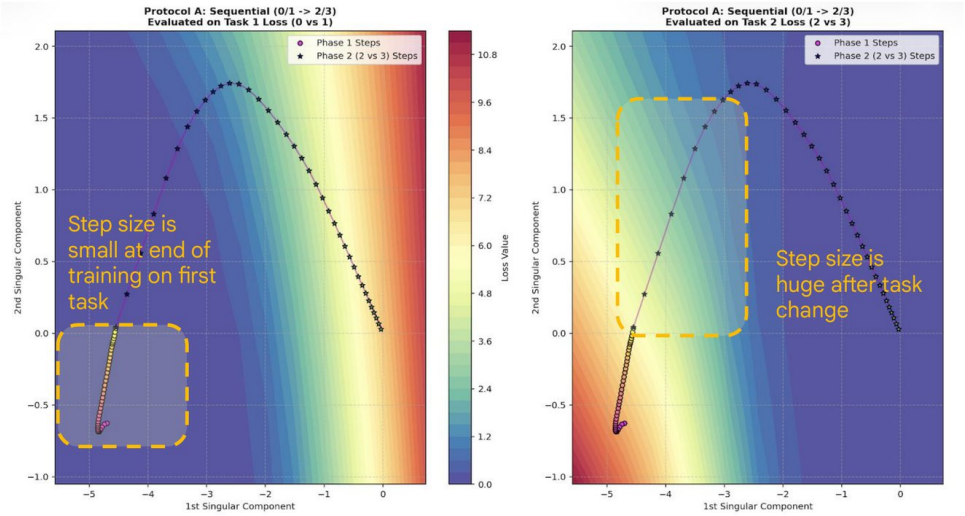

Figure 7.2: Why plasticity is lost immediately rather than gradually, from the Continual Learning lecture. The same optimisation trajectory is drawn over the task-one loss surface (left) and the task-two loss surface (right), projected onto two singular components. Steps are tightly packed at the end of task one, boxed on the left, and become very large as soon as the loss switches to task two, boxed on the right. Adam's slow-decaying second moment still reflects the small gradients of the previous task, so the effective step size overshoots.

Why a large step is specifically destructive. A large step does not merely overshoot a minimum. It overshoots the receptive field of the nonlinearity: the preactivations for a unit are pushed outside the range where the activation function is responsive, given the feature covariance the network had built up. That is how ReLUs die. It also explains why the remedy is not simply a smaller learning rate but control of the geometry, and it connects to a neat observation in the lecture on parameter norm: rotating a large parameter vector by a given angle requires a much larger perturbation than rotating a small one, so unchecked norm growth makes the network progressively harder to move in function space (Figure 7.3).

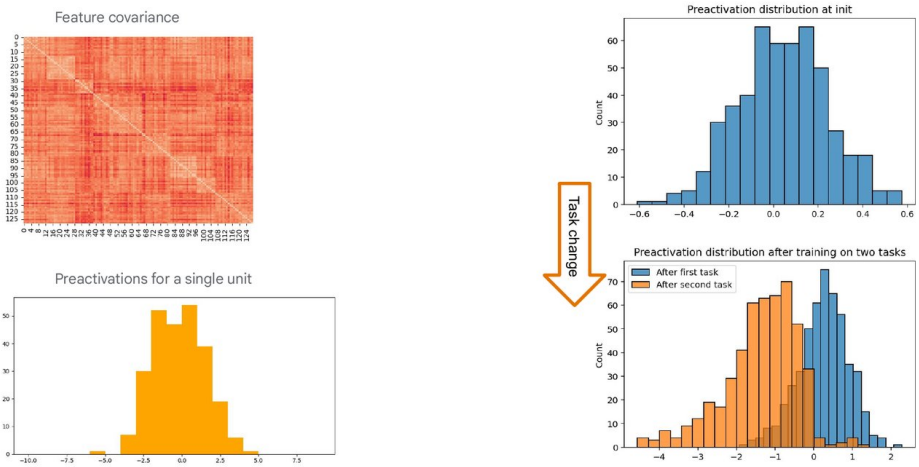

Figure 7.3: How a large step kills a unit, from the Continual Learning lecture. Left, the feature covariance the network has built up and the preactivation histogram for one unit. Right, the same unit's preactivation distribution at initialisation, centred on zero and narrow, and after training on two tasks: the second task's distribution, in orange, has shifted almost entirely below zero. Once preactivations leave the range where the activation responds, the unit stops passing gradient, which is what "the ReLUs are dead" means in practice.

The fixes are boring. What Lyle calls the swiss cheese approach is three complementary interventions: control parameter norm growth, normalise preactivations, and avoid regressing on ill-conditioned targets. The lesson is stated as use normalisation layers and do not let parameters get too big. With these in place continual training stabilises to the degree that the network can memorise effectively unlimited random labels of MNIST, and the same treatment stabilises reinforcement learning. A separate and non-catastrophic route to plasticity loss is warm-starting [1], tied to primacy bias: early in training a model makes large changes relative to its current features, later only small ones. The proposed remedy is to re-warm the effective learning rate above the threshold at which the model still makes nontrivial changes to its features.

Notation

The chapter follows the lecture's informal usage: $\mathcal{T}$ for a task, plasticity discussed through the effective step size rather than a single symbol. Where the lecture refers to the empirical NTK matrix it means the Gram matrix of per-example gradients at convergence, used as a diagnostic of conditioning rather than as a theoretical object.

Why it matters

This is the chapter that answers the closing slide of Chapter 2, where Chandar named non-i.i.d. learning as the frontier without saying what breaks. The answer here is specific: the optimiser's state is stale and the nonlinearity's receptive field is finite. It also stands in productive contrast with Pascanu (Chapter 8), who reaches for the same problem from the direction of credit assignment and argues something stronger, that the i.i.d. assumption is what makes gradient-based learning work at all. Lyle is the more optimistic of the two: her position is that most plasticity loss is avoidable with best practices, and that resetting and distilling remains available as a fallback.

NOT FROM THE LECTURE

The personal notes single out "catastrophic forgetting" as the takeaway and ask how to train networks online. Lyle's answer, read carefully, is partly deflationary: where the data can be stored, storing it is the right move, and the interesting problem only begins under a memory constraint. Which regime a project is in decides whether the literature applies at all, since it is largely written for the constrained case and most applications are not.

8 To Be Figured Out: Inductive Bias, Non-Stationarity and Linear Recurrence

Razvan Pascanu • *Mila and Google DeepMind*

The title above is the deck's own, and it is accurate: this is a survey of open problems rather than a result. Its spine is an argument about inductive bias: there is no prior-free machine learning system, every choice in building one is a bias, and the useful question is therefore not whether to have biases but which ones buy the most. Pascanu develops that in three registers. Conceptually, the i.i.d. assumption is itself a bias doing hidden work, because gradient descent resolves credit assignment by a tug-of-war between examples and that only works when the data is shuffled. Structurally, depth and architecture are biases with quantifiable payoffs. Practically, state space models and the linear recurrent unit buy capability from structure instead of scale, and by the end most of that advantage has evaporated into a hybrid.

Key ideas

Why look beyond i.i.d. at all. Four reasons, none of them about benchmark scores. Scaling is unsustainable, argued with data-centre water and electricity figures rather than FLOPs. The world is bigger than the agent: models have finite capacity, the world is large and includes the people modelling it, so continual learning is in a sense unavoidable. Learning on the fly reframes the goal from asymptotic performance to a tracking question, how well can we follow a moving target, which makes trade-offs such as compute against performance and robustness against performance explicit. And efficiency of learning, via Griffiths [25]: the profile of human intelligence is a consequence of human limitations, and AlphaGo's move 37 was available precisely because the machine does not decompose problems into tractable subproblems the way people do, sacrificing performance for efficiency. Pascanu's hypothesis is that trade-offs are necessary and that limitations should be exploited to shape the loss surface.

The i.i.d. assumption is doing hidden work. Gradient-based credit assignment inside a function approximator proceeds by tug-of-war dynamics between competing examples, which requires the data to be i.i.d. Two consequences are offered as hypotheses, not facts: there is no explicit knowledge composition, and scale becomes a crutch that makes learning possible at all. Failure of credit assignment is then what catastrophic forgetting *is*, rather than a separate phenomenon. The natural inference, that curricula should therefore help, does not survive contact [60, 66], and the lecture explains why exploiting this is not easy: we do not know how to measure what we care about when building a representation, and learning lacks compositionality and locality.

No prior-free learning, and depth as a quantifiable prior. Priors, regularisers and inductive biases all restrict or reshape the search space, and since every step in constructing a system is such a choice, the King Midas framing is that the choice cannot be declined. Depth is the cleanest example of a bias with a countable payoff: Zaslavsky's theorem on hyperplane arrangements [73] bounds how a single layer partitions input space, and composing layers multiplies regions, which is the counting argument behind the number

of linear regions of a deep network [48]. On optimisation the lecture is deflationary about the folklore. High-dimensional non-convex loss surfaces are dominated by saddles rather than bad minima [13], so the deep learning myth, that networks can be dropped in anywhere and optimisation behaves as if convex, holds only if the right protocol is used. Initialisation and the choice of optimiser qualitatively alter which solution is reached, which Pascanu argues should be leveraged instead of treated as nuisance; and a loss surface of sufficient dimension contains every low-dimensional pattern [12], which is a warning about reading too much into loss-landscape visualisations.

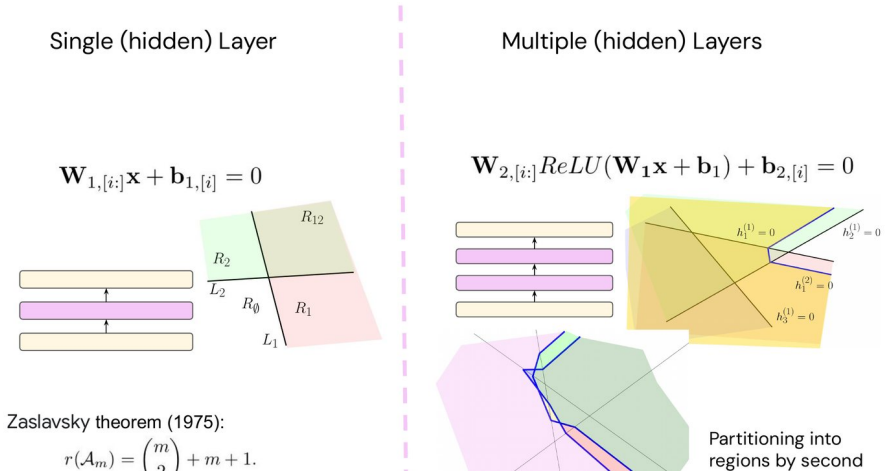

Figure 8.1: Depth as a bias with a countable payoff, from the To Be Figured Out lecture. A single hidden layer cuts input space with hyperplanes $\mathbf{W}_{1,[i:]}\mathbf{x} + \mathbf{b}_{1,[i]} = 0$, and Zaslavsky's formula gives the number of resulting regions. A second layer's boundaries $\mathbf{W}_{2,[i:]}\,\mathrm{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_{2,[i]} = 0$ are piecewise linear, bending as they cross the first layer's regions, so the count multiplies rather than adds. This is the origami picture of why depth buys expressivity.

State space models, and why they are hard to reason about. The second half opens with a hardware question: are transformers optimal? The worry is inference cost, usually framed as the quadratic cost of attention, and the comparison the lecture draws is per-step: recurrence is $O(1)$, attention $O(N)$. State space models are the community's main answer, and Pascanu describes them as a variant of linear RNNs. An SSM models the sequence in continuous time, relies on a deterministic HiPPO initialisation, and is then discretised; the payoff is that the discretised system can be read either as a recurrence, which makes inference fast, or as a convolution, which makes training fast. That dual view is the whole appeal.

His objection is methodological, not empirical. There are many variants (S4 [27], S4D, H3, Mamba [26]), several discretisation schemes such as bilinear and zero-order hold, it is unclear how changes to the surrounding block interact with the SSM's parameterisation, and for the better-performing SSMs the continuous-time story becomes strenuous. The question he poses is how to disentangle which ingredient is actually doing the work.

The LRU as an ablation, not a new model. The linear recurrent unit is the answer to that question, derived in six steps [54]. Start from a transformer backbone, because the MLP supplies the expressivity and therefore the recurrent block is allowed to be simple. Remove the nonlinearity from the recurrence, which costs less than expected: linear RNNs followed by nonlinear projections can approximate any finite sequence-to-sequence map, with the linear recurrence compressing the sequence and the MLP

approximating the decompressor. The caveat is stated, that this construction can only exhibit fading memory. Linearity then permits diagonalisation, so a complex diagonal $\mathbf{\Lambda}$ is learned directly and the recurrent computation collapses, and stability becomes a statement about eigenvalues: inside the unit disk gives exponentially decaying sine waves, outside is unstable. Vanishing gradients are the mirror image, since eigenvalues near the origin decay within a few steps, so initialisation samples an annulus that excludes the origin, typically $r_{\min} = 0.9$ and $r_{\max} = 0.999$ (Figure 8.2). Stable parameterisation and normalisation, the latter needed to control hidden-state growth, complete the recipe.

The results that matter here are the negative ones. HiPPO is not necessary. Continuous time is not necessary. Complex numbers may be doing nothing more than providing a compact representation, since a symmetric real system can perform similar computations at the cost of a larger state. In other words, most of the conceptual apparatus that SSM papers carry is detachable, and what remains is a diagonal linear recurrence with a sane spectrum. The lecture then descends to hardware, noting a roughly eightfold gap between HBM and VMEM transfer rates on the TPU profiled, and that a linear RNN performs an associative scan and so parallelises across time despite being a recurrence.

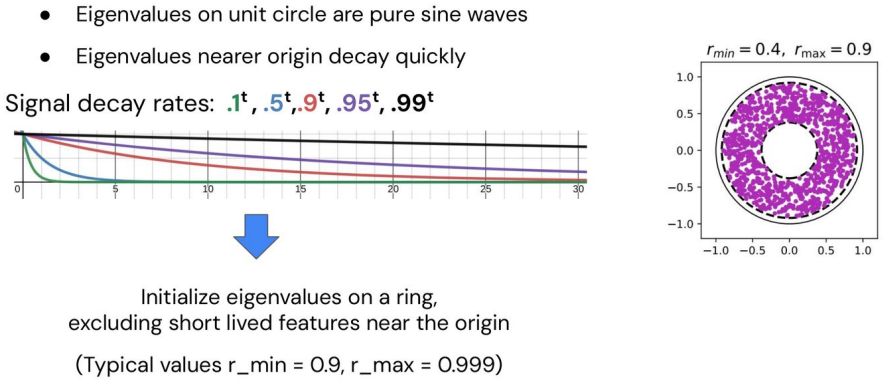

Figure 8.2: Step three of the linear recurrent unit derivation, from the To Be Figured Out lecture. Left, signal decay r^t for r from 0.1 to 0.99: an eigenvalue of modulus 0.5 has effectively forgotten its input within five steps, while 0.99 persists past thirty. Right, the consequence for initialisation, eigenvalues sampled from an annulus rather than a disk so that no unit starts with a memory too short to be useful. The annulus drawn uses $r_{\min} = 0.4$, $r_{\max} = 0.9$; the values recommended in practice are 0.9 and 0.999.

The scorecard, and the hybrid that actually wins. Having built the case, Pascanu reports that pure recurrence does not straightforwardly win. On language S4D did not work out of the box and an LSTM was better; gating it helped. What does work is a hybrid, and the reasoning is that SSMs and linear attention share the two properties that matter, a finite state and linear scaling in sequence length, while being complementary in what they capture: the SSM models the whole sequence, linear attention captures local information. The recipe given is a block pattern repeating two SSM blocks to one linear-attention block, and the configuration reported as state of the art at scale is roughly 75% linear attention with 25% global softmax attention retained (Figure 8.3). As models scale, behaviour and limitations converge, so the remaining gains are situational: online inference, and scaling down by distillation. The closing summary keeps both halves: biases are unavoidable, continual learning may be the next change of perspective, scale might work but is not the only option, transformers rely on a factorisation of simple components that must be scaled, and attention might not be all we need.

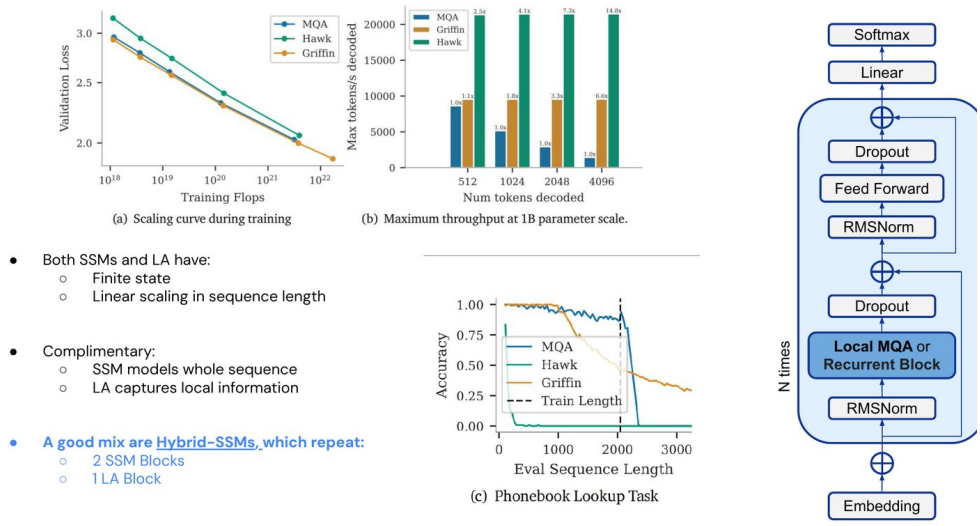

Figure 8.3: What actually wins, from the To Be Figured Out lecture. Left, the case for hybrids: state space models and linear attention share a finite state and linear scaling in sequence length, and are complementary because the SSM models the whole sequence while linear attention captures local structure. Middle, the evidence, with matched scaling curves, an order-of-magnitude throughput advantage at 1B parameters, and a phonebook-lookup task where the pure recurrent variants degrade past their training length. Right, the resulting block, alternating a recurrent or local-attention block with feed-forward layers.

Notation

$\mathbf{\Lambda}$ denotes the diagonal matrix of recurrence eigenvalues, r their modulus, t discrete time. The lecture uses θ for parameters as elsewhere in this book, and writes θ_0, $\theta_{\text{generalize}}$ and θ_{overfit} for points in parameter space when discussing capacity, which is retained here in prose rather than symbols.

Why it matters

Read this immediately after Chapter 7, because Lyle and Pascanu disagree in an interesting way. Lyle treats plasticity loss as a set of fixable engineering faults in the optimiser and the architecture; Pascanu treats the same territory as evidence that gradient-based credit assignment is structurally dependent on i.i.d. data, which is a much stronger claim and would mean the fixes are palliative. The linear-recurrence material also connects to Chapter 11: both lectures end up at the same place, that the binding constraint is memory bandwidth rather than arithmetic, and both reach it from opposite directions. The personal note to "check the MAMBA paper" belongs here [26], with the caution that Pascanu's point is precisely that Mamba's continuous-time framing is not what makes it work.

NOT FROM THE LECTURE

Pascanu's deck borrows three slides from Kyunghyun Cho on language modelling, and cites the statistics view that firing a linguist improves the recogniser. Set against Chapter 6, where Cho argues for learning the estimator rather than modelling the process, the two speakers are making the same argument from different ends: Cho says the generative process need not be modelled, Pascanu says some prior about it is unavoidable. Both are right, and what is left over is the question of which biases are cheap to state and expensive to learn.

9 ML for Mathematics

Matija Tapušković • *University of Oxford*

Tapušković is a mathematician rather than a machine learning researcher, and the talk asks one question: can machine learning discover new mathematics? The answer is organised into three strands, learning from examples, searching in language space, and formalisation, and the recurring test applied to each is the one a mathematician would apply: where does the machine's output re-enter mathematics? Not whether a metric improved, but whether the result becomes a theorem, a conjecture someone will chase, or merely a number. The conclusion running through all three strands is that the search is rarely the bottleneck. Building a cheap, exact and ungameable evaluator is the research.

Key ideas

Strand I, learning from examples, works when attribution is interpretable. The flagship result is on Kazhdan–Lusztig polynomials. A permutation becomes a directed graph, an interval $[x, y]$ in the Bruhat order becomes a subgraph, and the combinatorial invariance conjecture of Lusztig and Dyer says isomorphic unlabelled interval graphs give the same polynomial. A message-passing GNN was trained on the bare interval graph to predict the coefficients of $P_{x,y}$, and gradient attribution flagged particular edges as overrepresented [14]. Those flagged edges pointed at a decomposition: a recipe computes $P_{x,y}$ from a split of the interval into two smaller pieces, though the recipe needs labels, and the new conjecture is that any valid split agrees, which would imply combinatorial invariance (Figure 9.1). A variant of that conjecture has since been proved for broad families. The point is that the network's value was not its accuracy but the fact that its attribution was legible enough to become a mathematical statement.

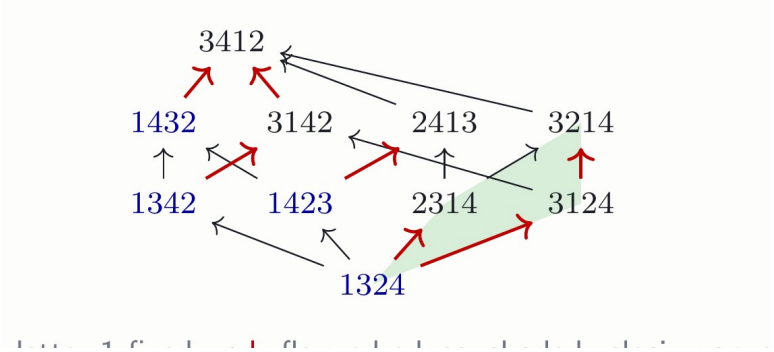

Figure 9.1: The output that re-entered mathematics, from the ML for Mathematics lecture. An interval in the Bruhat order on permutations, drawn as a directed graph. Red marks the edges that gradient attribution on the trained GNN flagged as overrepresented, blue marks vertices whose first letter is fixed, and the shaded region is the closing square of the split those flagged edges suggested. This picture, not the model's accuracy, is what became a conjecture.

The counterexample in the same strand is instructive. On elliptic curves $E : y^2 = x^3 + Ax + B$, the rank in $E(\mathbb{Q}) \cong E(\mathbb{Q})_{\text{tors}} \oplus \mathbb{Z}^{\text{rank}(E)}$ is not directly computable, so the experiment represented each curve by its sequence of point counts $a_p = (p+1) - N_p(E)$

and ran logistic regression on $(a_2, a_3, \dots) \in \mathbb{Z}^{500}$ to separate rank 0 from rank 1, reaching over 97% accuracy and beating known heuristics [30]. The expert reaction quoted on the slide is the lesson: what has been learned is the sign in the functional equation of the associated L-function, not the rank. The model was right for a reason that made it uninformative, and the data came from an archive rather than a generator, which limits what could be probed.

A worked case study in why data, not method, is the constraint. The Feynman-integral material is the most detailed part of the lecture and reads as a cautionary tale told against the speaker's own field. The question is whether weight drop, conjecturally equivalent to $c_2 = 0$, is visible in elementary graph features. On 1,958 graphs up to 11 loops with 272 positives and 24 features, AUC was 0.90. Then the deflation begins. The top feature restates a known theorem, that a 3-vertex cut forces a product and hence $c_2 = 0$; remove those positives and AUC falls to 0.85. The 165 survivors form only 7 families with labels constant within a family, so the classifier is largely recognising disguised siblings; hold out whole families and AUC falls to 0.74. In the full catalogue of 283 irreducible families up to 10 loops, exactly 7 have the drop, and those 7 are conspicuous anyway, with automorphism groups roughly twenty times larger than typical and degenerate, often integer, spectra. With 7 positives the finding is a description, not a rule. Two further constraints are named: labels are genuinely expensive, since at 11 loops c_2 is known at only four primes and the next prime can cost hours per graph; and GNNs are the wrong tool here because ϕ^4 graphs are 4-regular, so a message-passing network is blind by 1-WL to exactly the signal that works, which points at the geometric deep learning toolkit as the fix (Chapter 12).

Strand II, search in language space, when there is no dataset. The reframing is that a scoring function is enough to enable search, and the key move is not to search the space of objects but the space of programs that generate them, so that a program inherits the score of its output and an LLM proposes variants. The lineage given runs from extremal-combinatorics search through PatternBoost to FunSearch and AlphaEvolve. The flagship evaluation is 67 problems, hours each, almost all of the form improve a bound on a constant, with a scorecard of 20 new, 39 matched and 8 fell short [24]. The authors' own framing is quoted rather than softened: none of the results is individually important, every one could have been found with standard tools, and the point is time; the tool beats the literature where the literature is thin and matches or trails it where it is deep. The concrete highlight is Kakeya and Nikodym sets over finite fields, where the previous Kakeya record in dimension three was $\frac{1}{4}p^3 + \frac{7}{8}p^2 + O(p)$ and AlphaEvolve found $\frac{1}{4}p^3 + \frac{7}{8}p^2 - \frac{1}{8}$ for $q = p \equiv 1 \bmod 4$, a two-line formula over quadratic residues. For Nikodym sets the machine's idea, delete surfaces rather than points, was no better than random as used, and a human repaired it by hand into a genuine improvement. The recurring observation is that the expertise moved into the evaluator.

Strand III, certainty. A proof assistant checks that a term inhabits a type, so compiling a statement establishes well-formedness, not truth, and the proof term is what must be constructed. The motivating question is sharp: would any expert read a 150-page proof of a major conjecture handed over by a language model? Formal infrastructure is uneven, with mathlib at roughly 2.3M lines built over nine years, covering approximately an undergraduate degree and jagged beyond it. The competition trajectory given is 2024 AlphaProof and AlphaGeometry at silver with the kernel as evaluator, 2025 Deep Think at gold in plain English graded by humans, and 2026 AxiomProver at 42 out of 42, formal end to end, roughly 8,000 lines of Lean in about 25 hours. The research-level example

closes the loop: for minimal Kakeya sets in $\mathbb{F}_p^3$, AlphaEvolve found the construction, Deep Think proved it and AlphaProof certified it, the full pipeline closed, but only because every ingredient was already in mathlib. In dimension 4 elliptic curves appear and the certification fails; for Nikodym sets the construction was too tangled to analyse and the final proof was entirely human. The frontier of machine certification runs out precisely where the deep objects begin.

Notation

S_n is the symmetric group, $P_{x,y}(q)$ a Kazhdan–Lusztig polynomial, E an elliptic curve with a_p its trace of Frobenius at p, c_2 the Feynman-graph invariant whose vanishing is weight drop, and $\pi(x)$ the prime counting function.

Why it matters

This chapter is a rigorous account of what it takes for a machine learning result to count as a contribution in a field with an external standard of correctness, which makes it the sharpest methodological lecture of the week. Its central complaint, that the evaluator is the research and the search is not the bottleneck, transfers directly to reinforcement learning (Chapter 5), where RLVR is exactly the claim that a cheap, exact, ungameable reward changes what is learnable. The Feynman-graph observation that 4-regularity makes message passing blind is a concrete instance of the expressivity limits that Chapter 12 addresses by construction.

NOT FROM THE LECTURE

The three strands are an escalating standard of evidence, and the same ladder can be applied to ordinary machine learning work. Strand I asks whether a model's attribution is legible enough to state as a claim. Strand II asks whether the score is faithful enough that optimising it means something. Strand III asks whether the result can be checked without trusting the producer. Most machine learning papers only clear the second bar, and clear it against a metric far less exact than a Lean kernel.

10 Spectral Analysis of Directed Acyclic Graphs

Isidora Stanković • *Faculty of Electrical Engineering, University of Montenegro*

One obstruction, one construction, one proof that the construction costs nothing. Graph signal processing supplies a Fourier transform by diagonalising the adjacency matrix, and on a directed acyclic graph that machinery fails completely, because a DAG's adjacency matrix is nilpotent and therefore has every eigenvalue equal to zero. Stanković's answer is to add a return edge to close the graph into a cycle, which restores a spectrum, and then to insert enough zero-valued vertices along that return path that the feedback it introduces cannot reach the original vertices within the filter's horizon. The spectrum comes back and the filter output on the original graph is unchanged.

Key ideas

Why anyone should care about DAG spectra. DAGs are everywhere in machine learning: computation graphs, causal and Bayesian networks, and autoregressive models, where causal attention is precisely a DAG. The graph Fourier transform is the basis for spectral GNNs and for graph filtering, so if it degenerates on DAGs then an entire toolkit is unavailable on a class of graphs we use constantly.

The setup, in one paragraph. For a directed graph $G = (V, E)$ with $|V| = N$ and adjacency matrix $\mathbf{A}$, a graph signal is $\mathbf{x} = [x(1), \ldots, x(N)]^{\mathsf{T}}$. The shift operator is the adjacency matrix itself, and a linear graph system is a polynomial in it,

$$H = \sum_{s=0}^{S} h_s \mathbf{A}^s, \qquad y = H\mathbf{x} = \sum_{s=0}^{S} h_s \mathbf{A}^s \mathbf{x},$$

where $\mathbf{A}^s$ describes propagation along directed paths of length s. This is graph convolution: $\mathbf{Ax}$ is one message-passing hop and the h_s are exactly the coefficients a spectral GNN learns. If $\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1}$ is diagonalisable, the GFT is $X = \mathbf{V}^{-1}\mathbf{x}$ and the system acts multiplicatively in the spectral domain, $H(\mathbf{\Lambda}) = \sum_s h_s \mathbf{\Lambda}^s$.

The obstruction is structural, not numerical. Topologically order a DAG's vertices and its adjacency matrix becomes strictly upper triangular, which the lecture notes has the same shape as a causal attention mask. A strictly upper triangular matrix is nilpotent, $\mathbf{A}^K = \mathbf{0}$ for large enough K, so every eigenvalue is zero and $\mathbf{\Lambda} = \mathbf{0}$. The consequence is not a loss of precision but a total collapse:

$$H(\mathbf{\Lambda}) = \sum_{s=0}^{S} h_s \mathbf{\Lambda}^s = h_0 \mathbf{I},$$

so every graph mode has the identical gain h_0, spectral components become indistinguishable, frequency ordering disappears, and the adjacency-based GFT loses all interpretability. What survives is vertex-domain propagation: $\mathbf{A}^s\mathbf{x}$ is still perfectly meaningful. The spectrum, not the graph, is what breaks.

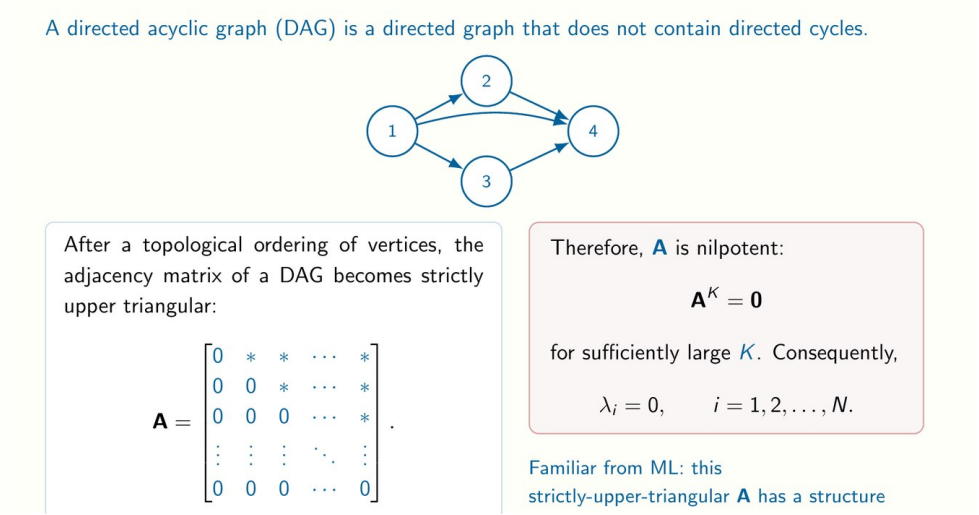

Figure 10.1: Where the obstruction comes from, from the Spectral Analysis of DAGs lecture. Topologically ordering a DAG's vertices makes the adjacency matrix strictly upper triangular, which forces $\mathbf{A}^K = \mathbf{0}$ and hence $\lambda_i = 0$ for every i. The lecture's own aside, bottom right: this is the same matrix structure as a causal attention mask, so every decoder-only transformer has the same degenerate spectrum.

Zero-padding, and the analogy that motivates it. The construction is borrowed from classical DFT practice (Figure 10.2). A finite signal has no spectrum until it is extended periodically, which introduces circular wrap-around; zero-padding is how signal processing prevents that wrap-around from contaminating the interval of interest. The graph version runs in three steps. A directed path has no spectrum because propagation does not wrap around. Add a return edge from sink to source and the graph becomes a cycle, whose adjacency eigenvalues are $\lambda_k = e^{j2\pi(k-1)/N}$ with complex-exponential eigenvectors, so graph spectral analysis on the cycle coincides exactly with the classical DFT. But the return edge is feedback. So insert $M \geq S$ zero-valued vertices along it, which delay that feedback past the horizon of any filter of order S.

The guarantee, and why the cycle is not cheating. The augmented graph is cyclic on purpose; the original object is still a DAG and the larger graph is a spectral embedding used only for the transform. The formal statement is that for $0 \leq s \leq S$,

$$\left(\mathbf{A}_{\mathrm{ZP}}^{s}\mathbf{x}_{\mathrm{ZP}}\right)_{1:N} = \mathbf{A}^{s}\mathbf{x}, \qquad \mathbf{x}_{\mathrm{ZP}} = \begin{bmatrix} \mathbf{x} \\ \mathbf{0} \end{bmatrix},$$

so filter outputs on the original vertices satisfy $y_{\mathrm{ZP}}(1{:}N) = y$ exactly. After padding, $\mathbf{A}_{\mathrm{ZP}}$ has determinant ± 1 and so is invertible, meaning no eigenvalue is zero; the eigenvalue angle $\omega_k = \arg\{\lambda_k\}$ becomes the graph frequency, with slow modes at small angles and fast modes at large ones, and a Fourier-like interpretation is restored. Diagonalisability additionally needs distinct eigenvalues, which the talk checks exhaustively rather than assuming: over all connected DAGs on 7 vertices, a bare sink-to-source edge gives 98.4% diagonalisable and one padding vertex gives 99.97%; on 8 vertices, 99.0% and, with two padding vertices, 99.91%. The residual cases resolve by giving the added edge weight 0.5.

General DAGs, and the cost. A single return edge suffices only when one directed Hamiltonian path visits every vertex. A general DAG has several sources and sinks and no such path, so the pipeline is two-stage: connect the DAG so a Hamiltonian path exists,

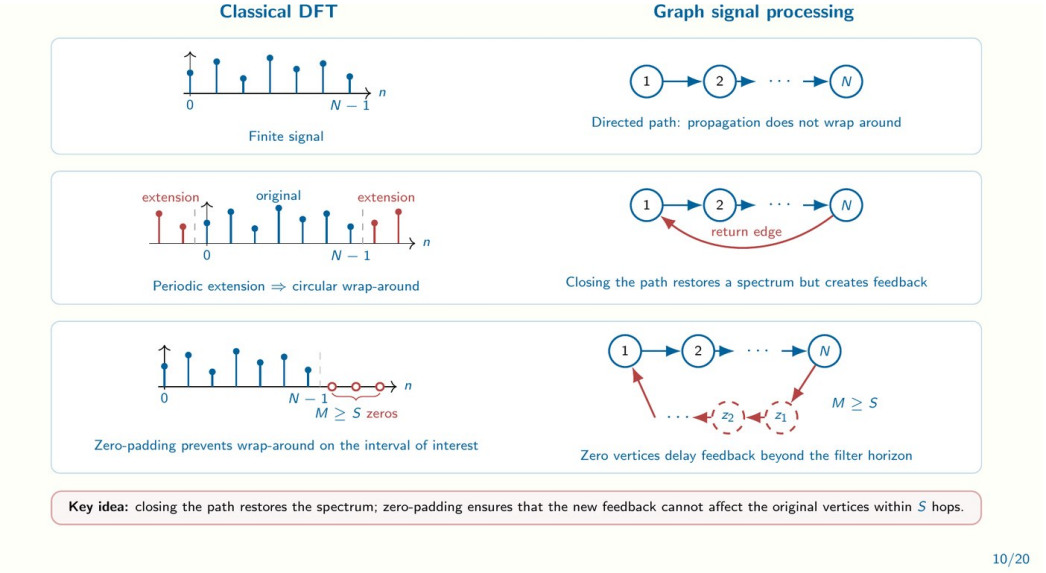

Figure 10.2: The construction and the analogy it comes from, from the Spectral Analysis of DAGs lecture. Left column, classical DFT: a finite signal has no spectrum, periodic extension supplies one at the cost of circular wrap-around, and zero-padding pushes that wrap-around outside the interval of interest. Right column, the graph version step for step: a directed path does not wrap around, a sink-to-source return edge closes it and restores a spectrum but introduces feedback, and $M \geq S$ zero vertices $z_1, z_2, \ldots$ along the return path delay that feedback beyond the horizon of any order-S filter.

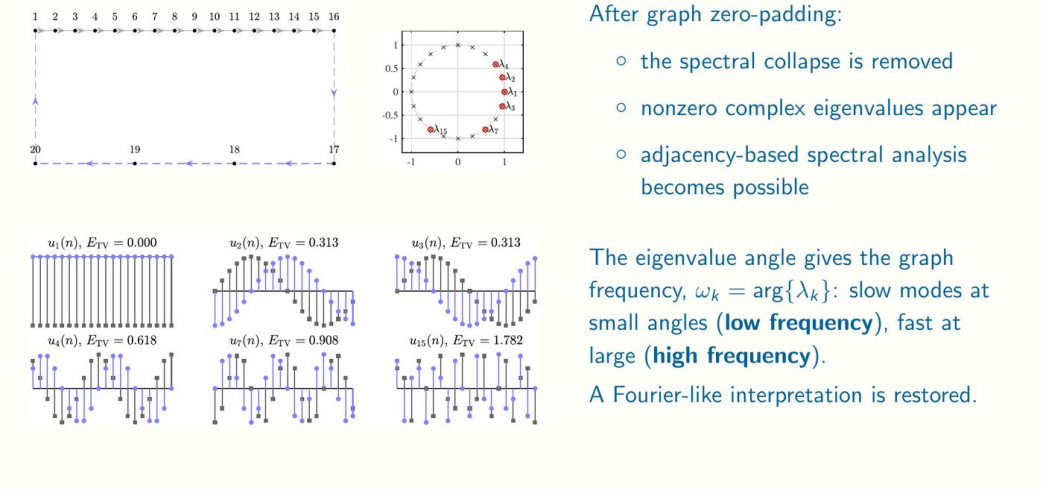

Figure 10.3: What is recovered, from the Spectral Analysis of DAGs lecture. Top left, a zero-padded graph on 16 original vertices with the padding path closing it. Top middle, the resulting eigenvalues, now nonzero complex numbers spread around the unit circle rather than all collapsed at the origin. Bottom left, the corresponding eigenvectors ordered by total variation E_{TV}, from a constant mode at 0.000 to a rapidly oscillating one at 1.782, which is exactly the low-to-high frequency ordering a Fourier basis is supposed to provide.

then pad every added edge. The overhead is linear, $N_{\mathrm{ZP}} = N + (K+1)M$ vertices for K connecting edges with M padding vertices each. The worked application is denoising: a graph filter designed in the restored spectral domain preserves the smoothest spectral components while attenuating noise-dominated ones, taking a synthetic example from $\mathrm{SNR}_{\mathrm{in}} = -6.99$ dB to $\mathrm{SNR}_{\mathrm{out}} = 6.21$ dB (Figure 10.4).

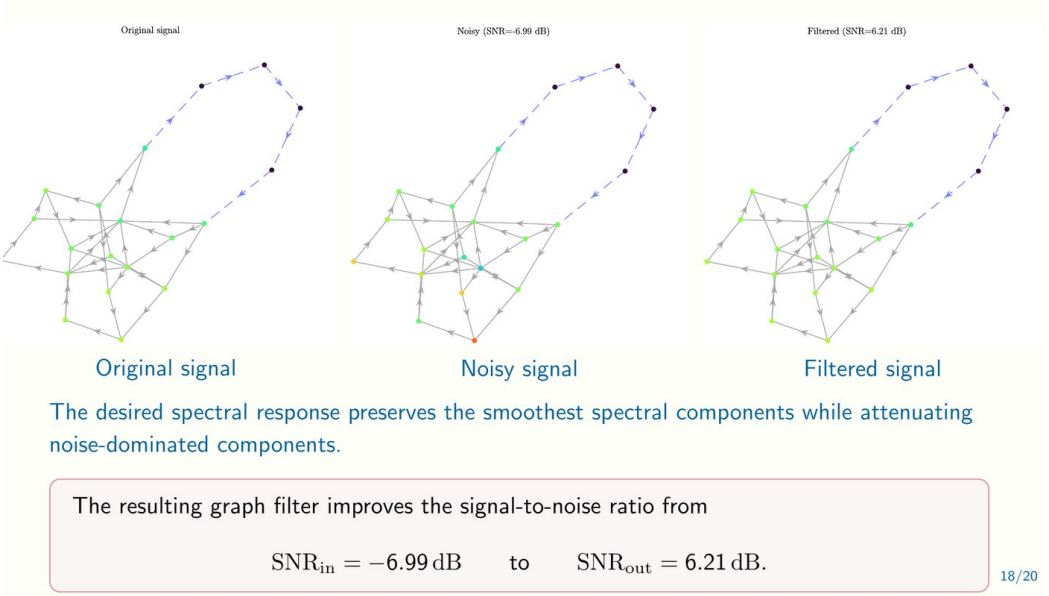

Figure 10.4: The payoff, from the Spectral Analysis of DAGs lecture. The same DAG carrying a smooth signal, that signal with noise added, and the result of applying a low-pass graph filter designed in the restored spectral domain. Vertex colour is signal value. The filter is only expressible because zero-padding gave the DAG a spectrum to design in, and it recovers roughly 13 dB.

Notation

$\mathbf{A}$ is the adjacency matrix, $\mathbf{A}_{\mathrm{ZP}}$ its zero-padded counterpart, $\mathbf{V}$ and $\mathbf{\Lambda}$ the eigenvector and eigenvalue matrices, $\mathbf{x}$ a graph signal, h_s filter coefficients, S the filter order and M the number of padding vertices. Note the collision with Chapter 5, where S is the state space; here S is a filter order, following each speaker's own usage.

Why it matters

The lecture's own hook, that a strictly upper triangular adjacency matrix looks like a causal attention mask, is the connection to chase. It means every decoder-only transformer is a DAG whose spectral analysis is currently unavailable, and this construction is a candidate route to it. That places the talk directly between Dieleman's spectral view of diffusion (Chapter 4), which treats the spectrum as the object being generated, and Veličković's geometric deep learning (Chapter 12), where the graph Fourier transform underwrites spectral GNNs. It is also a good model of a small, complete contribution: an obstruction, a fix, a proof that the fix is free, and an exhaustive check of the one assumption that could fail.

NOT FROM THE LECTURE

The construction's soft spot is not the spectrum but the requirement of distinct eigenvalues. Instead of assuming genericity, the talk enumerates all 2^{21} connected DAGs on eight vertices and reports the failure rate. A complete enumeration on small instances costs an afternoon.

11 Computationally-Efficient Learning

Dan Alistarh • *ISTA*

Alistarh opens with the bitter lesson, that general methods leveraging computation win by a large margin, and then turns it against itself. If computation is the scarce resource, the discipline of making methods computationally efficient is not opposed to the bitter lesson but is its consequence, which he later names the bitter lesson squared: designing computationally efficient methods to make large models more computationally efficient. The evidence for scarcity is a scissors between demand and supply, and the rest of the lecture is one mathematical idea applied to closing it. That idea is that quantization error should be measured in loss space rather than weight space, weighted by the inverse Hessian, which turns out to be a 1990 result rediscovered at scale.

Key ideas

The gap, in three numbers. Between 2020 and 2025, on public data, computation used to train state-of-the-art models rose by roughly 50,000 times and inference computation by roughly 300 times, while the capability of the best available GPU rose by only about 20 (Figure 11.1). The slide labels the resulting divergence a 2500-fold gap. Since hardware is visibly not closing it, algorithms must, and the concrete stakes are given in model sizes: Llama-4 Maverick at 400 billion total parameters, 800 GB at half precision, 17 billion active across 128 experts, trained on 22 trillion tokens for about 6 million GPU hours. The lecture also puts a cost on inference rather than training, quoting a Google study from August 2025: a median generative query consumes 0.24 Wh, 0.03 g of CO_2e, described as nine seconds of television, and 0.26 mL of water, five drops. Two caveats come with it: medians hide heavy prompts, and market-based carbon accounting undercounts location-based.

The mathematics is about the loss, not the weights. Quantization is just a particular weight perturbation, $\text{Compress}(\mathbf{w}) = \mathbf{w} + \delta\mathbf{w}$. Taylor expanding the loss,

$$\mathcal{L}(\mathbf{w} + \delta\mathbf{w}) \approx \mathcal{L}(\mathbf{w}) + \nabla_{\mathbf{w}}\mathcal{L} \cdot \delta\mathbf{w} + \tfrac{1}{2}\,\delta\mathbf{w}^{\top}\mathbf{H}\,\delta\mathbf{w},$$

and since the weights are trained, $\nabla_{\mathbf{w}}\mathcal{L} \approx \mathbf{0}$, so the quantity actually being minimised is the quadratic form $\frac{1}{2}\delta\mathbf{w}^{\top}\mathbf{H}\delta\mathbf{w}$ subject to $\delta\mathbf{w}$ having the admissible form $\delta w_i = w_i - Q(w_i)$ for quantizer Q. For a single weight the optimum is

$$\text{LossImpact}(w_i) = \frac{\left(w_i - Q(w_i)\right)^2}{2\,[\mathbf{H}^{-1}]_{ii}}, \qquad \delta\mathbf{w}^{*} = -\,\frac{w_i - Q(w_i)}{[\mathbf{H}^{-1}]_{ii}}\;\mathbf{H}^{-1}_{:,i}.$$

Two things follow. The cost of rounding a weight is not its rounding error but that error divided by the corresponding inverse-Hessian entry, so weights differ enormously in how much they may be moved. And the correction $\delta\mathbf{w}^{*}$ spreads compensation across the other weights, which is why good quantizers adjust what they have not yet touched. The lecture attributes this lineage explicitly to Optimal Brain Damage and Optimal Brain Surgeon, from 1990 and 1993 [29, 44], and to its own scaling in the Optimal Brain Compressor.

Why it becomes practical at LLM scale. GPTQ and SparseGPT minimise a layer-wise loss $\left\|\mathbf{Y}_\ell - \mathbf{X}_\ell\widehat{\mathbf{W}}_\ell\right\|^2$ whose Hessian is $\mathbf{X}_\ell\mathbf{X}_\ell^{\top}$, and the observation that unlocks everything

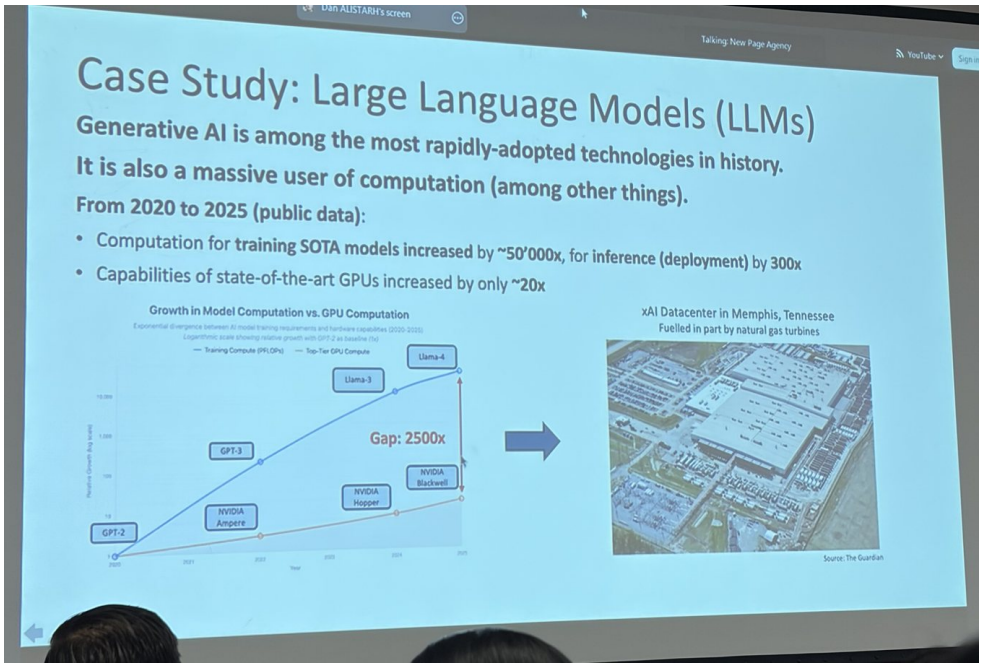

Figure 11.1: The argument for the whole lecture, from the Computationally-Efficient Learning slides, photographed during the talk. Training compute per state-of-the-art model and top-tier GPU capability are plotted on a log scale with GPT-2 as baseline. The two curves diverge from GPT-2 through GPT-3 to Llama-3 and Llama-4 while the hardware line stays nearly flat across Ampere, Hopper and Blackwell, leaving the annotated 2500-fold gap. The photograph on the right is the xAI data centre in Memphis, partly gas-fuelled.

is that this Hessian is constant with respect to the weights, so it can be precomputed once at the start of compression [21, 22]. The algorithm then sweeps columns of $\mathbf{W}_\ell$: quantize column j in saliency order within a block, compute the error, and update the columns to the right of j using $\delta \mathbf{w}^*$. The supporting machinery is a family of linear time and space estimators of second-order information at block, layer and global granularity, with the observation that different models need different granularities.

What the large study actually found. A million-plus individual task and model evaluations [41] yield three findings the lecture reduces to one slide. Eight-bit quantization is essentially lossless. All compression approaches land within roughly 1% of baseline on easy benchmarks but degrade more on harder ones, reasoning in particular. And larger models, 70B and above, are easier to compress and compress more stably across tasks. The study deliberately restricted itself to schemes with hardware support, W4A16 with INT4 and W8A8 dynamic per-token with INT8 and FP8, which is what makes the numbers actionable rather than academic.

From measurement to prediction. The last technical move is to stop measuring the damage and predict it. The linearity theorem claims that under stated assumptions the effect of quantization on perplexity is approximately linear in the relative layer-wise squared error,

$$\mathbb{E}\big[\mathrm{PPL}(\widehat{\mathbf{W}})\big] \approx \mathrm{PPL}(\mathbf{W}^*) + \sum_{\ell=1}^{L} \alpha_\ell \, t_\ell^2,$$

with α_ℓ absolute per-layer constants and t_ℓ the L_2 error at layer ℓ [46]. The disclaimer is stated on the slide rather than buried: the theory holds only for methods that do not

update weights, so it covers AWQ but not GPTQ. That is a real limitation, since GPTQ's whole mechanism is the compensating update.

Notation

$\mathbf{w}$ are trained weights, $\delta\mathbf{w}$ the quantization perturbation, Q the quantizer, $\mathbf{H}$ the Hessian and PPL perplexity. Note that $\mathbf{H}$ here is a Hessian, whereas $\mathbf{H}$ in Chapter 3 is a centring matrix, and the two are kept visually distinct for exactly that reason.

Why it matters

This chapter pairs with the summer school's quantization tutorial, which implements round-to-nearest and GPTQ against the same mathematics and exhibits the 4-bit failure the findings predict. It also converges with Pascanu (Chapter 8) on the same diagnosis from a different direction: Pascanu profiles memory bandwidth and concludes the recurrent block should be cheap, Alistarh reduces bit-width so the weights fit in faster memory, and both are treating arithmetic as abundant and data movement as scarce. The lottery ticket citation is shared with Chapter 2 under a different interpretation, which is noted there.

NOT FROM THE LECTURE

Alistarh takes the bitter lesson, which is normally deployed as an argument against clever methods, and derives from it a mandate for clever methods, on the grounds that if compute is what matters then compute efficiency is what matters.

12 A Modern Tutorial on Geometric Deep Learning

Petar Veličković • *Google DeepMind and University of Cambridge*

The organising analogy is historical. In the nineteenth century geometry had fragmented into a zoo of competing geometries, and Klein's Erlangen Program unified them by asking, for each one, which group of transformations it leaves invariant. Veličković argues deep learning in the 2020s is in the same position, a zoo of architectures with no principle organising them, and proposes the same remedy: specify the domain the data lives on and the group of symmetries acting on it, then derive the admissible layers. The payoff claimed is not aesthetic. For several architectures the derivation is tight enough to show that they *must* have the form they do, and the same recipe extends to domains where nobody has intuition, such as spheres and meshes.

Key ideas

Invariance, equivariance, and why the second is the useful one. A function is invariant if the symmetry leaves its output unchanged, and equivariant if the output transforms correspondingly. The definition given is that for $f : \mathcal{X} \to \mathcal{Y}$, a group $\mathfrak{G}$, and representations ρ_1 and ρ_2 acting on the domain and codomain respectively, f is $\mathfrak{G}$-equivariant when

$$f \circ \rho_1(g) = \rho_2(g) \circ f \qquad \text{for all } g \in \mathfrak{G},$$

so applying the symmetry before f is the same as applying it after. Invariance is the special case $\rho_2 = \mathrm{id}$. The lecture's point is that invariance is too strong to build with, because an invariant layer must collapse the whole input to a single output and thereby throws structure away. The construction is therefore a stack of equivariant layers with an invariant readout only at the end. Whether something is a symmetry is a property of the task, not of the data: rotation is a symmetry for cats versus dogs and emphatically not for digit recognition. Groups organise these symmetries, group actions turn them into operations on the domain, and representations turn the actions into matrices, with $SO(2)$ rotations and S_3 permutations as the running examples.

The recipe. Data lives on a domain Ω, an $N \times N$ grid, a set, a graph, and the data itself is a signal $\mathcal{X} : \Omega \to \mathcal{C}$. The key structural observation is that an action on the domain induces an action on signals. Given that, the building blocks are linear equivariant maps composed with local nonlinearities, and the derivation of each architecture is just working out which linear maps are equivariant to the group in question. The lecture then works this through for what it calls the four Gs: graphs, grids, groups and geometric graphs, with manifolds and gauges as the finale, following the structure of the companion book [3].

Graphs give message passing, with no freedom left. Permutation equivariance plus a locality requirement forces the layer to be a function of a node and its neighbourhood, and the result is the message-passing form

$$\mathbf{h}_u = \phi\Big(\mathbf{x}_u, \bigoplus_{v \in \mathcal{N}_u} \psi(\mathbf{x}_u, \mathbf{x}_v)\Big),$$

where ψ is the message function, $\bigoplus$ the aggregation and ϕ the update (Figure 12.1). The only requirement on $\bigoplus$ is permutation invariance, so sum, max or mean all qualify, and

message and update are the only parametric parts. The lecture is explicit that this is derived rather than designed: locality is presented as possibly the only way to satisfy equivariance without collapsing.

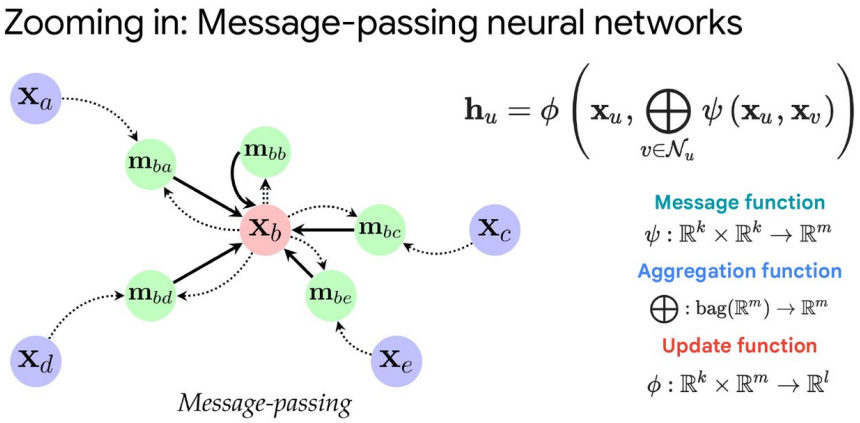

Figure 12.1: The message-passing layer as derived in the Geometric Deep Learning tutorial. Node b receives a message $\mathbf{m}_{bv}$ from each neighbour, the messages are combined by a permutation-invariant aggregator, and the update produces $\mathbf{h}_u$. Only ψ and ϕ carry parameters; the aggregator's only job is to make the layer indifferent to neighbour ordering.

Transformers are graph neural networks. The most provocative section argues that treating LLMs as sequence models is a category error. Nodes are tokens, $V = \{1, \ldots, N\}$, features are token embeddings, and the causal mask is precisely a neighbourhood definition, $v \in \mathcal{N}_u \iff v \leq u$. Dot-product self-attention is then a message function in two parts, with the softmax denominator handled by an aggregator that accumulates both numerator and normaliser and an update that divides, so the whole block is an MPNN on a complete or causally-masked graph [36], with graph attention networks [67] credited on the slide as the attentional precursor. The practical value is diagnostic: once a transformer is a GNN, its known pathologies become graph pathologies, and the lecture names overmixing and oversquashing as exactly the phenomena the GNN literature has studied. Both connect back to the DAG structure of causal attention in Chapter 10.

Grids give CNNs, and the derivation is tight. A grid is a graph with more regularity, which narrows the permutations that matter down to the translations. Collecting the parameters of a translation-equivariant linear map into a matrix forces that matrix to be circulant, and every circulant matrix is a convolution. The conclusion is stronger than motivation: all linear translation-equivariant maps on a grid are convolutions, which is why CNNs must have the form they do. The same style of argument recovers gating in recurrent networks from invariance to time warping, which is a satisfying derivation of GRU and LSTM structure from a symmetry rather than from engineering.

Groups, and building for domains that resist intuition. Generalising convolution from translations to an arbitrary group $\mathfrak{G}$ gives group convolution [9], and the subtlety flagged is that after the first layer the signals live on $\mathfrak{G}$ rather than on the original domain, which is what makes depth work. Spheres are the worked example: $\Omega = \mathbb{S}^2$ with 3D rotation

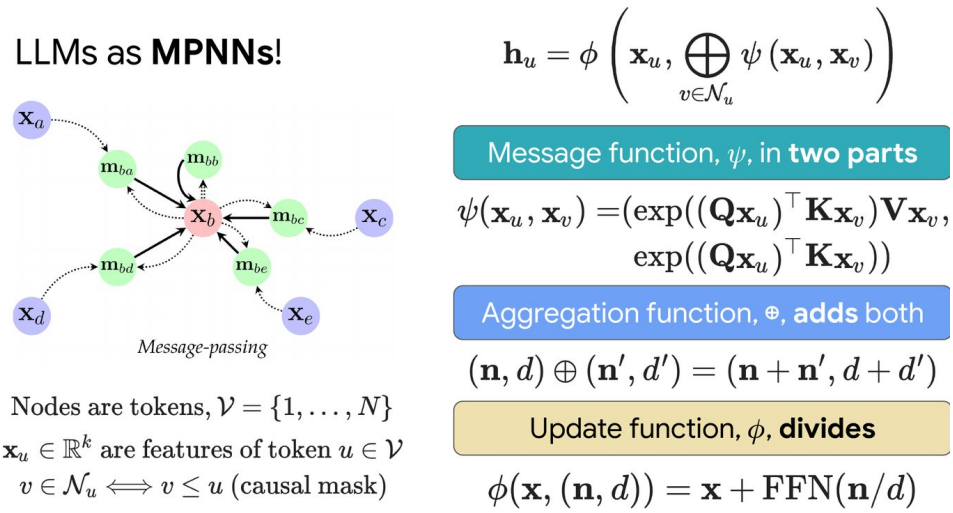

Figure 12.2: A transformer block written as message passing, from the Geometric Deep Learning tutorial. Nodes are tokens, $\mathcal{N}_u$ is fixed by the causal mask $v \leq u$, and the block decomposes exactly into the three MPNN components. The message function carries two parts, the attention numerator $\exp((\mathbf{Q}\mathbf{x}_u)^\top \mathbf{K}\mathbf{x}_v)\mathbf{V}\mathbf{x}_v$ and its denominator $\exp((\mathbf{Q}\mathbf{x}_u)^\top \mathbf{K}\mathbf{x}_v)$; the aggregator sums both componentwise, $(\mathbf{n}, d) \oplus (\mathbf{n}', d') = (\mathbf{n} + \mathbf{n}', d + d')$; and the update performs the softmax division and the residual feed-forward step, $\phi(\mathbf{x}, (\mathbf{n}, d)) = \mathbf{x} + \mathrm{FFN}(\mathbf{n}/d)$.

symmetry, layer one maps $\mathbb{S}^2 \to SO(3)$ and subsequent layers act on $SO(3)$ [8]. The applied highlight is TacticAI (Figure 12.3), where the relevant symmetry is the dihedral group D_2, an approximate symmetry of a football pitch under reflection along each axis [69]. This is the result the personal notes record as predicting corner outcomes several seconds ahead, and the reason it belongs in this lecture is that the pitch symmetry is approximate, so the equivariance is a modelling choice rather than a fact.

Geometric graphs, manifolds and gauges. If the domain does not float in a vacuum but is embedded in space, the relevant group is $E(3)$, roto-translation and reflection, combined with permutation equivariance. The cheap route to guaranteed invariance is to message pass only on invariant quantities such as pairwise distances; vector-valued features need genuinely equivariant updates instead. Manifolds push this furthest. A 2D manifold is locally like $\mathbb{R}^2$, so the work happens in the tangent space $T_u\Omega$, movement along the surface goes through the exponential map $\exp_u X$, and tangent vectors at different points are compared by parallel transport $\Gamma_{u \to v} X$. Defining convolution in the tangent space then runs into the central difficulty: the choice of gauge, the local frame, is entirely arbitrary, and even on a sphere no consistent global choice exists. Requiring gauge equivariance directly means conditioning across pairs of arbitrarily related gauges, which the lecture calls very hard; parallel transporting features first simplifies it. The resolution is deflationary and rather satisfying: what results is a GNN with extra precomputations and angle-conditioned parameters, or as the slide puts it, just a graph with extra structure. AlphaFold 2 carries the same ideas, maintaining a local frame per residue [37] (Figure 12.4).

Notation

Ω is the domain, $\mathcal{X}$ a signal on it, $\mathfrak{G}$ a group with elements g, ρ a representation, $\mathcal{N}_u$ the neighbourhood of node u, and ψ, $\bigoplus$, ϕ the message, aggregation and update

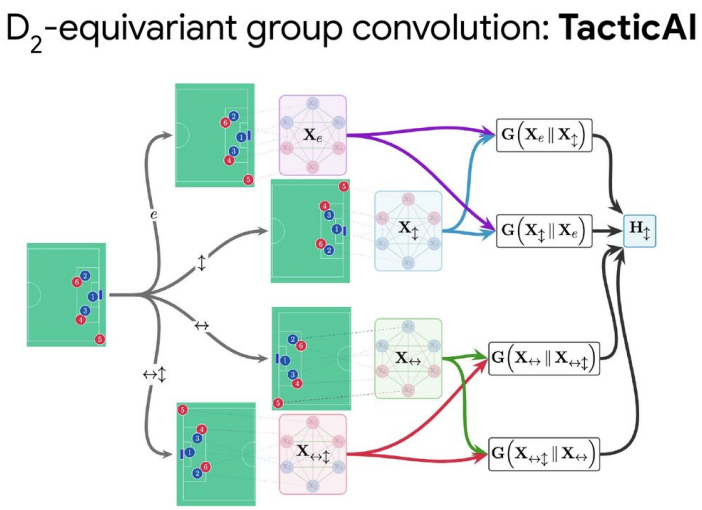

Figure 12.3: Group convolution in the wild, from the Geometric Deep Learning tutorial. A corner-kick configuration is transformed by each element of the dihedral group D_2, reflection along the pitch's long axis, its short axis, and both, giving four views. Each is encoded, the encodings are combined pairwise, and the result is pooled into a D_2-equivariant representation, so the model cannot give a different answer merely because the play happened on the other side of the pitch.

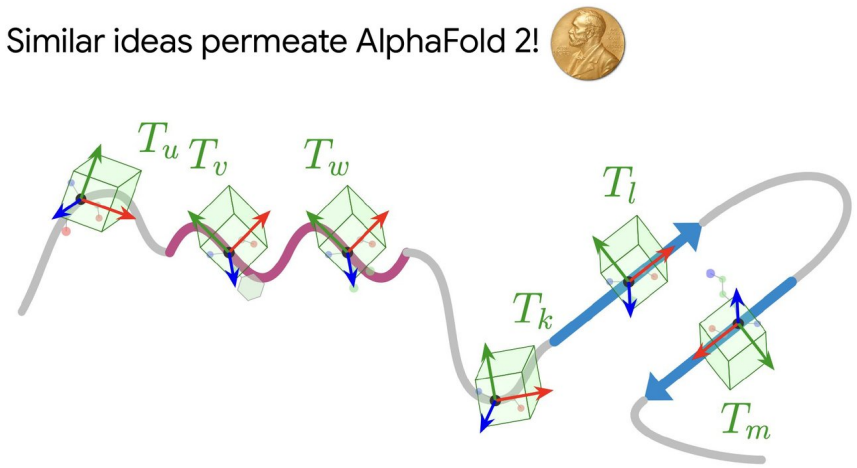

Figure 12.4: Where the gauge machinery reappears, from the closing of the Geometric Deep Learning tutorial. A protein backbone with a local coordinate frame $T_u, T_v, T_w, \ldots$ attached to each residue. Reasoning about relative geometry means transporting information between these frames, which is the same problem as choosing gauges on a manifold. This is the slide the personal notes record but whose image attachment is missing from disk; it is recovered here from the deck.

functions. The slides write ϕ as ϕ and ψ as ψ, and that usage is kept here. Isola uses ϕ for a representation in Chapter 3, which is an unavoidable collision between two speakers' conventions.

Why it matters

This lecture retroactively organises several others. Cho's Set Transformer++ (Chapter 6) is a permutation equivariant architecture derived exactly the way this lecture prescribes, without either speaker citing the other. The Feynman-graph failure in Chapter 9, where ϕ^4 graphs are 4-regular and so message passing is blind by 1-WL, is a case where this toolkit is the named fix. And the transformers-as-GNNs claim is the bridge to Chapter 10: if a causally-masked transformer is a GNN on a DAG, then its adjacency spectrum collapses, and zero-padding is a candidate for recovering it.

NOT FROM THE LECTURE

Two follow-ups from the personal notes belong here. The mathematical prerequisites are published separately as a long companion on the group- and representation-theoretic background, recorded in those notes as `https://arxiv.org/abs/2508.02723`, and the course home is `https://geometricdeeplearning.com`, with the MIT Press book dated 2026 on the closing slide. The dependency runs one way: nothing in the four Gs requires that background in order to be *used*, but the derivations, which are the actual content of the lecture, do.

Part II

Synthesis

13 Cross-cutting Themes

Six ideas recurred across the week, and in three cases two speakers reached the same territory from opposite directions or disagreed outright. A disagreement between people who both know the field marks an open question rather than a gap in the notes, so those three are the ones to dwell on.

1. The i.i.d. assumption, and a real disagreement about how bad it is

Chandar closes the opening lecture (Chapter 2) by naming the frontier without explaining it: current practice optimises the i.i.d. case, while true learning is sequential, online and continual. Three later lectures pick that up, and they do not agree.

Lyle (Chapter 7) treats it as an engineering fault with an identifiable mechanism. Plasticity is lost because Adam's second-moment estimate is stale after a task change, so the effective step size explodes, and because a large step pushes preactivations outside the receptive field of the nonlinearity. Both are fixable by normalising, controlling parameter norm, and avoiding regression on ill-conditioned targets, with resetting and distilling as a fallback. Her verdict on forgetting is even more deflationary: if the data can be kept, the problem is easy.

Pascanu (Chapter 8) makes a structural claim instead. Gradient-based credit assignment works by tug-of-war dynamics between competing examples, and that mechanism *requires* shuffled data. On this reading catastrophic forgetting is not a side effect but is what failed credit assignment looks like, and Lyle's fixes are palliative because they stabilise training without giving the learner explicit knowledge composition. He offers as hypotheses that there is no compositional knowledge in these models and that scale is a crutch making learning possible at all.

Deac's tutorial material (Chapter 5) is the third data point and it supports Pascanu quietly: experience replay exists precisely because consecutive transitions are correlated and SGD assumes independence. Replay is continual learning solved by refusing to be in the continual setting, which is exactly Lyle's advice, and it is also an admission that nobody knows how to learn from a correlated stream directly.

The disagreement is testable, which is what makes it interesting; Chapter 14 sketches an experiment that would separate the two positions.

2. Symmetry imposed versus symmetry measured

Veličković (Chapter 12) and Isola (Chapter 3) are doing the same thing from opposite ends. Veličković specifies the symmetry group first and derives which layers are admissible, strongly enough that CNNs are shown to be the only linear translation-equivariant maps on a grid. Isola takes trained networks and measures, after the fact, which transformations their representations are invariant to, then argues the right equivalence is isometry because distance is what downstream tasks consume. One prescribes the invariance, the other discovers it.

The most striking evidence that this is one idea rather than two is that Cho (Chapter 6) derives a permutation-equivariant architecture, a transformer with positional embeddings removed, from exactly Veličković's reasoning, in a lecture about statistics, without either speaker referencing the other. The same argument arriving independently in three lectures is some evidence it is not incidental.

3. Where knowledge gets injected

Three lectures address the same question: if a model cannot learn everything from data, where does the rest come from?

Cho's answer is to stop modelling the generative process. Inference need not invert generation, so an estimator can be trained directly on problems with known answers, at the price of making identifiability relative to the prior over problems, the hypothesis space and the optimiser. Pascanu's answer is the mirror image: there is no prior-free machine learning system, every construction step is a bias, so the question is never whether to encode knowledge but which encoding pays. And Tapušković (Chapter 9) locates it in the evaluator, arguing that in the 67-problem study search was never the bottleneck and that building a cheap, exact, ungameable score *is* the research.

Isola sits awkwardly across this. His lecture uses fixed metrics such as CKA to establish that models converge, but his stated opinion is that there is no future for distance functions that are not learned. Taken together with Cho, that points somewhere uncomfortable: if both the model and the yardstick are fitted, convergence results become hard to state without circularity.

4. The spectrum as the design surface

Three unrelated lectures turn out to be doing spectral engineering. Dieleman (Chapter 4) explains why diffusion works on images by observing that natural images have a decaying power spectrum while Gaussian noise is flat, so the noise schedule is effectively a frequency schedule and sampling recovers bands coarsest-first. Stanković (Chapter 10) cannot filter on a DAG at all until a spectrum is restored, because nilpotency collapses every eigenvalue to zero. And Pascanu's linear recurrent unit is designed entirely in the spectral domain, with the eigenvalue modulus setting the memory horizon and the initialisation annulus chosen so that no unit begins with a memory too short to matter.

In all three the eigenvalues are not a diagnostic applied afterwards; they are the thing being chosen.

5. Arithmetic is cheap, moving data is not

Alistarh (Chapter 11) and Pascanu converge on one conclusion from opposite directions. Alistarh's framing is a 2500-fold gap between compute demand and hardware capability over five years, and his remedy is to reduce bit-width so that weights occupy less of the scarce resource. Pascanu profiles a TPU, finds roughly an eightfold gap between HBM and VMEM transfer rates, and concludes the recurrent block should be cheap because the memory traffic dominates. Neither is optimising FLOPs. Both treat arithmetic as abundant and data movement as the constraint. The first question in any efficiency work is what has to move, not how many operations there are.

6. Transformers are not what we call them

Three lectures reframe the same architecture, and the reframings are compatible. Veličković argues transformers are graph neural networks and not sequence models, with tokens as nodes and the causal mask as a neighbourhood, which makes overmixing and oversquashing inherited graph pathologies rather than novel LLM problems. Stanković notes in passing that a causal attention mask is a strictly upper triangular matrix, hence a DAG, hence spectrally degenerate. Pascanu argues transformers are a factorisation into simple components that must be scaled, and that attention might not be all we need, while

also reporting honestly that the hybrid alternative retains 25% global softmax attention because pure recurrence loses on language.

Read together they say the architecture is better understood as a message-passing operator on a particular graph than as a sequence model, and that its known limitations follow from that graph.

NOT FROM THE LECTURE

One theme is conspicuous by its absence. Nobody argued for scale as a sufficient answer. Alistarh opens with the bitter lesson and immediately derives a mandate for efficiency from it; Pascanu says scale might work but is not the only option; Tapušković reports that the tool beats the literature only where the literature is thin. That is a summer school's selection effect as much as a fact about the field, but it is consistent across eleven speakers.

14 Research Directions

Each item below is attributed to where it came from. Items marked **flagged** were named as open by the speaker. Items marked **inferred** are editorial, drawn from placing two lectures next to each other, and should be treated as untested conjecture rather than as anything the speakers endorsed.

Problems the speakers explicitly left open

Measuring what we care about when building a representation. Flagged, Pascanu (Chapter 8). He argues that non-i.i.d. structure cannot be exploited because we do not know how to measure representation quality during training, and separately because learning lacks compositionality and locality. The measurement problem is the tractable half: it is a concrete request for a training-time signal that predicts downstream representation quality.

Evaluating continual learning at all. Flagged, Lyle (Chapter 7). She states the meta-problem plainly and neither option is acceptable: a pre-specified task sequence is artificial, a naturally continual application is uncontrolled. Any result in the field inherits this weakness, so a defensible evaluation protocol would be a contribution in itself.

In-context learning as a continual learning mechanism. Flagged, Lyle. Her closing slide calls this an exciting paradigm that brings its own challenges, without saying which. Given the rest of her lecture, the obvious question is whether in-context adaptation escapes the plasticity problem or merely relocates it into the context window.

What else admits a learned estimator. Flagged, Cho (Chapter 6). Asked directly on a slide. He has done population standard deviation, mutual information, Bayesian predictive distributions and causal effects. The programme's requirement is a simulator of problems with known answers, which is the real filter on what comes next.

Whether a world model aligns the way vision and language models do. Flagged, Isola (Chapter 3). Posed as an open question on a slide. His convergence results are for vision and language; whether a model trained on interaction and dynamics lands on the same kernel is untested and would discriminate between the three hypotheses he offers about what "same" means.

Generalisation in diffusion models, and flow maps. Flagged, Dieleman (Chapter 4). His closing slide lists what he did not cover: generalisation versus memorisation, integrating diffusion models via flow maps [18], reward-based steering and finetuning, and multimodal generative models. The flow-maps thread is also flagged in the personal notes.

Quantized training, and extending the linearity theorem. Flagged, Alistarh (Chapter 11). He names quantized training as the frontier and says big open questions sit at the intersection of representations, optimisation and compute. There is also a precise gap on the slide: the linearity theorem holds only for methods that do not update weights, so it covers AWQ but not GPTQ, which is awkward because GPTQ's compensating update is its whole mechanism.

Building faithful evaluators, and learning a Galois action. Flagged, Tapušković (Chapter 9). His stated conclusion is that search is not the bottleneck and constructing a cheap, exact, ungameable score is the research. He also asks directly whether the action of the Cosmic Galois Group on Feynman Mellin motives can be learned, and observes that machine certification stops exactly where the deep objects begin, which is a diagnosis of where formalisation effort should go.

Gauge equivariance without parallel transport. Flagged, Veličković (Chapter 12). Conditioning across arbitrarily related pairs of gauges is described as very hard, and the working solution sidesteps it by transporting features first. Whether the harder formulation buys anything is open.

Why real-world reinforcement learning fails. Flagged, Deac (Chapter 5). Her own live bandit collected zero votes over 53 rounds, and the closing slide enumerates the causes: reward confounded with content, non-stationarity, sparsity, noise, varying turnout, delayed and batched feedback, and a false independence assumption across action dimensions. That list is a better research agenda than most papers' future-work sections.

Directions inferred here, not reported

Give causally-masked transformers a spectrum. Inferred, from Stanković and Veličković. Stanković notes that a causal attention mask is strictly upper triangular and therefore nilpotent, and Veličković argues a transformer *is* a GNN on that graph. Putting the two together, every decoder-only transformer is a graph whose adjacency spectrum has collapsed, and her zero-padding construction is a candidate for restoring it: add a return path with enough zero tokens that the induced feedback cannot reach real tokens within the depth of the network. Whether the restored spectrum says anything useful about attention is entirely untested, and the construction was designed for graph filters rather than learned attention, so this may not survive contact.

An experiment that would separate Lyle from Pascanu. Inferred. Lyle claims plasticity loss is a fixable optimiser and architecture fault; Pascanu claims gradient-based credit assignment structurally needs i.i.d. data. Apply Lyle's full remedy, normalisation, norm control, avoiding ill-conditioned targets, and then test not whether training remains stable, which she has shown, but whether knowledge *composes*: does a model trained on tasks A then B solve compositions of A and B better than one trained on B alone? Lyle's account predicts that stability suffices; Pascanu's predicts that composition still fails. No such experiment appears in either lecture.

A learned representational similarity metric. Inferred, from Cho and Isola. Isola says unlearned distance functions have no future; Cho shows how to learn an estimator whenever problems with known answers can be simulated. Representational similarity is a candidate: train an estimator on pairs of representations related by known transformations, with the transformation as the label. The hazard is the circularity flagged in Chapter 13: convergence results measured with a fitted metric are hard to interpret, and confronting that directly might be the more valuable output.

Model the compensating update in the linearity theorem. Inferred, from Alistarh. The theorem's disclaimer excludes weight-updating methods. GPTQ's update is not arbitrary though: it is the closed-form $\delta\mathbf{w}^*$ from the inverse Hessian. That suggests the theorem

might extend by carrying the update through the per-layer error term rather than by treating it as unmodelled noise. Speculative, and whether the assumptions survive has not been checked.

NOT FROM THE LECTURE

Every item in the second list is a guess made by putting two lectures side by side, and none of them was checked against the literature. Anyone who finds that one of them has already been done, or is already known not to work, is better informed than these notes are.

15 Worth Revisiting

Thirteen things from the week that reward a second pass, in no particular order. Some are readings, some are exercises, and a few are loose ends that nobody has tied up.

1. **The 4-bit failure in the quantization tutorial**, in the section headed 8-bit RTN. Round-to-nearest quantization is essentially lossless at 8 bits and visibly destroys the same model at 4. Watching that happen before reading Chapter 11 is what makes the inverse-Hessian derivation feel necessary rather than decorative.
2. **Activation steering**, exercises 9 to 11 of the mechanistic interpretability tutorial. A steering vector is built from the contrast between two sets of prompts, added to the residual stream mid-generation, and the effect on the output is unmistakable. The whole method fits in three exercises and transfers to any model that can be wrapped in TransformerLens.
3. **Continual learning, read as a disagreement.** Chapter 7 and then Chapter 8. Lyle treats plasticity loss as a fixable fault in the optimiser and the architecture; Pascanu treats the same territory as evidence that gradient-based credit assignment structurally depends on i.i.d. data. Reading them in that order makes the gap between the two positions visible in a way neither lecture does alone.
4. **Dieleman on diffusion as spectral autoregression**, `https://sander.ai/2023/07/20/perspectives.html`. The slide version, Figure 4.3, compresses the argument to four panels. The post carries the derivation, and it is the most reusable idea in the diffusion lecture: the noise schedule is a frequency schedule.
5. **PPO and GRPO, implemented in that order**, Part 2 of the reinforcement learning tutorial. PPO is built first with a critic, at full memory cost, and GRPO then removes the critic in a few lines by standardising rewards within a sampled group. The ordering is what makes GRPO's appeal legible rather than merely stated.
6. **The Kazhdan–Lusztig experiment** [14], with Chapter 9. Gradient attribution on a trained GNN flagged particular edges, those edges suggested a decomposition, and the decomposition became a conjecture. It is the clearest example available of a model's internals being legible enough to turn into a mathematical statement.
7. **The linear recurrent unit derivation, all six steps.** Orvieto et al. [54], with Chapter 8. An ablation dressed as an architecture, where the negative results are the substance: HiPPO is unnecessary, continuous time is unnecessary, and complex numbers may buy nothing beyond compactness. A better first reading on state space models than most explainers.
8. **CKA and the invariance argument around it** [39], with Chapter 3. A small formula with a great deal of thinking behind it. The transferable habit is to ask of any similarity measure which transformations it has been built to ignore, and whether those are the ones the downstream task ignores too.
9. **The eight-episode TD versus Monte Carlo exercise.** Slides 37 to 39 of the reinforcement learning deck, with Chapter 5. Five minutes with pen and paper, and it fixes the difference between fitting the observed data and fitting the model that data implies. Best attempted before looking at the answer.
10. **Computational identifiability.** Chapter 6, final section, and Bynum, Ranganath, and Cho [5]. Identifiability defined relative to a prior over problems, a hypothesis space and an optimiser, rather than absolutely. The idea from the week most likely to change how a problem is framed rather than how it is solved.
11. **Zero-padding applied to causal attention.** Chapter 10, with the inferred direction in Chapter 14. A causally-masked transformer is a DAG, so its adjacency spectrum

collapses. Whether Stanković's construction says anything useful about attention is untested, and cheap to sanity-check on a small mask.

12. **PECNET with a state space model**, presented at the poster session by Mustafa Abdullah Hakkoz for regional agricultural forecasting. Worth pairing with the state space material in Chapter 8, since the poster puts a hybrid to work on a real forecasting problem rather than on a benchmark.
13. **The geometric deep learning prerequisites**, https://arxiv.org/abs/2508.02723, with the course at https://geometricdeeplearning.com. The group- and representation-theoretic background is not needed to apply the four Gs, only to derive them. Time well spent for anyone intending to build an architecture from a symmetry rather than pick one off the shelf.

References

[1] J. T. Ash and R. P. Adams. "On Warm-Starting Neural Network Training". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Dated 2019 on the lecture slide. 2020. arXiv: 1910.08475 [cs.LG].

[2] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle. "Greedy Layer-Wise Training of Deep Networks". In: *Advances in Neural Information Processing Systems 19 (NIPS 2006)*. 2007, pp. 153–160.

[3] M. M. Bronstein, J. Bruna, T. Cohen, and P. Veličković. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. Course home: https://geometricdeeplearning.com. 2021. arXiv: 2104.13478 [cs.LG].

[4] J. Bruce et al. *Genie: Generative Interactive Environments*. 2024. arXiv: 2402.15391 [cs.LG].

[5] L. E. J. Bynum, R. Ranganath, and K. Cho. *Computational Identifiability*. 2026. arXiv: 2606.19361 [cs.LG].

[6] L. E. J. Bynum et al. *Black Box Causal Inference: Effect Estimation via Meta Prediction*. 2025. arXiv: 2503.05985 [cs.LG].

[7] M. Caron et al. "Emerging Properties in Self-Supervised Vision Transformers". In: *IEEE/CVF International Conference on Computer Vision (ICCV)*. 2021. arXiv: 2104.14294 [cs.CV].

[8] T. S. Cohen, M. Geiger, J. Köhler, and M. Welling. "Spherical CNNs". In: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1801.10130 [cs.LG].

[9] T. S. Cohen and M. Welling. "Group Equivariant Convolutional Networks". In: *International Conference on Machine Learning (ICML)*. 2016. arXiv: 1602.07576 [cs.LG].

[10] A. Conneau, G. Lample, M. Ranzato, L. Denoyer, and H. Jégou. "Word Translation Without Parallel Data". In: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1710.04087 [cs.CL].

[11] D. R. Cox. "The Regression Analysis of Binary Sequences". In: *Journal of the Royal Statistical Society: Series B* 20.2 (1958), pp. 215–242. DOI: 10.1111/j.2517-6161.1958.tb00292.x.

[12] W. M. Czarnecki, S. Osindero, R. Pascanu, and M. Jaderberg. *A Deep Neural Network's Loss Surface Contains Every Low-Dimensional Pattern*. 2019. arXiv: 1912.07559 [cs.LG].

[13] Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli, and Y. Bengio. "Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2014. arXiv: 1406.2572 [cs.LG].

[14] A. Davies et al. "Advancing Mathematics by Guiding Human Intuition with AI". In: *Nature* 600.7887 (2021), pp. 70–74. DOI: 10.1038/s41586-021-04086-x.

[15] DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. arXiv: 2501.12948 [cs.CL].

[16] S. Dieleman. *Perspectives on Diffusion*. Linked from the lecture slides. 2023. URL: https://sander.ai/2023/07/20/perspectives.html.

[17] S. Dieleman. *The Paradox of Diffusion Distillation*. Linked from the lecture slides. 2024. URL: https://sander.ai/2024/02/28/paradox.html.

[18] S. Dieleman. *Generative Modelling in Latent Space and Flow Maps*. Linked from the lecture's closing slide. 2026. URL: https://sander.ai/2026/05/06/flow-maps.html.

[19] S. Dieleman et al. *Continuous Diffusion for Categorical Data*. 2022. arXiv: 2211.15089 [cs.LG].

[20] J. Frankle and M. Carbin. "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks". In: *International Conference on Learning Representations (ICLR)*. 2019. arXiv: 1803.03635 [cs.LG].

[21] E. Frantar and D. Alistarh. "SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot". In: *International Conference on Machine Learning (ICML)*. 2023. arXiv: 2301.00774 [cs.LG].

[22] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: 2210.17323 [cs.LG].

[23] S. Fu et al. "DreamSim: Learning New Dimensions of Human Visual Similarity Using Synthetic Data". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. arXiv: 2306.09344 [cs.CV].

[24] B. Georgiev, J. Gómez-Serrano, T. Tao, and A. Z. Wagner. *Mathematical Exploration and Discovery at Scale*. 2025. arXiv: 2511.02864 [math.HO].

[25] T. L. Griffiths. "Understanding Human Intelligence through Human Limitations". In: *Trends in Cognitive Sciences* 24.11 (2020), pp. 873–883. DOI: 10.1016/j.tics.2020.09.001.

[26] A. Gu and T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2023. arXiv: 2312.00752 [cs.LG].

[27] A. Gu, K. Goel, and C. Ré. "Efficiently Modeling Long Sequences with Structured State Spaces". In: *International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2111.00396 [cs.LG].

[28] S. Gupta, S. Kansal, S. Jegelka, P. Isola, and V. Garg. *Canonicalizing Multimodal Contrastive Representation Learning*. 2026. arXiv: 2602.17584 [cs.LG].

[29] B. Hassibi and D. G. Stork. "Second Order Derivatives for Network Pruning: Optimal Brain Surgeon". In: *Advances in Neural Information Processing Systems (NIPS)*. Cited on the slide as Hassibi, Stork & Wolff, 1993. 1993, pp. 164–171.

[30] Y.-H. He, K.-H. Lee, T. Oliver, and A. Pozdnyakov. *Murmurations of Elliptic Curves*. Cited on the slide as He–Lee–Oliver–Pozdnyakov, 2022. 2022. arXiv: 2204.10140 [math.NT].

[31] G. E. Hinton, S. Osindero, and Y.-W. Teh. "A Fast Learning Algorithm for Deep Belief Nets". In: *Neural Computation* 18.7 (2006), pp. 1527–1554. DOI: 10.1162/neco.2006.18.7.1527.

[32] K. Hornik. "Approximation Capabilities of Multilayer Feedforward Networks". In: *Neural Networks* 4.2 (1991), pp. 251–257. DOI: 10.1016/0893-6080(91)90009-T.

[33] M. Huh, B. Cheung, T. Wang, and P. Isola. "The Platonic Representation Hypothesis". In: *International Conference on Machine Learning (ICML)*. 2024. arXiv: 2405.07987 [cs.LG].

[34] A. Hyvärinen. "Estimation of Non-Normalized Statistical Models by Score Matching". In: *Journal of Machine Learning Research* 6 (2005), pp. 695–709.

[35] R. Jha, C. Zhang, V. Shmatikov, and J. X. Morris. *Harnessing the Universal Geometry of Embeddings*. 2025. arXiv: 2505.12540 [cs.LG].

[36] C. K. Joshi. *Transformers are Graph Neural Networks*. Technical version of an article in The Gradient. 2025. arXiv: 2506.22084 [cs.LG].

[37] J. Jumper et al. "Highly Accurate Protein Structure Prediction with AlphaFold". In: *Nature* 596.7873 (2021), pp. 583–589. DOI: 10.1038/s41586-021-03819-2.

[38] J. Kirkpatrick et al. "Overcoming Catastrophic Forgetting in Neural Networks". In: *Proceedings of the National Academy of Sciences* 114.13 (2017), pp. 3521–3526. DOI: 10.1073/pnas.1611835114.

[39] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. "Similarity of Neural Network Representations Revisited". In: *International Conference on Machine Learning (ICML)*. 2019. arXiv: 1905.00414 [cs.LG].

[40] N. Kriegeskorte et al. "Matching Categorical Object Representations in Inferior Temporal Cortex of Man and Monkey". In: *Neuron* 60.6 (2008), pp. 1126–1141. DOI: 10.1016/j.neuron.2008.10.043.

[41] E. Kurtić, A. Marques, S. Pandit, M. Kurtz, and D. Alistarh. ""Give Me BF16 or Give Me Death"? Accuracy-Performance Trade-Offs in LLM Quantization". In: *Annual Meeting of the Association for Computational Linguistics (ACL)*. 2025. arXiv: 2411.02355 [cs.LG].

[42] R. J. LaLonde. "Evaluating the Econometric Evaluations of Training Programs with Experimental Data". In: *The American Economic Review* 76.4 (1986). Dated 1984 on the lecture slide, pp. 604–620.

[43] N. Lambert et al. *Tülu 3: Pushing Frontiers in Open Language Model Post-Training*. Cited on the deck as the RLVR reference. 2024. arXiv: 2411.15124 [cs.CL].

[44] Y. LeCun, J. S. Denker, and S. A. Solla. "Optimal Brain Damage". In: *Advances in Neural Information Processing Systems (NIPS)*. 1990, pp. 598–605.

[45] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. "Flow Matching for Generative Modeling". In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: 2210.02747 [cs.LG].

[46] V. Malinovskii, A. Panferov, I. Ilin, H. Guo, P. Richtárik, and D. Alistarh. "The Linearity Theorem: Rethinking Quantization Error Metrics for LLM Compression". In: *Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*. 2025. arXiv: 2411.17525 [cs.LG].

[47] M. Minsky and S. Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

[48] G. Montúfar, R. Pascanu, K. Cho, and Y. Bengio. "On the Number of Linear Regions of Deep Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2014. arXiv: 1402.1869 [stat.ML].

[49] S. Müller, N. Hollmann, S. P. Arango, J. Grabocka, and F. Hutter. "Transformers Can Do Bayesian Inference". In: *International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2112.10510 [cs.LG].

[50] A. Nichol et al. "GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models". In: *International Conference on Machine Learning (ICML)*. 2022. arXiv: 2112.10741 [cs.CV].

[51] A. van den Oord, N. Kalchbrenner, and K. Kavukcuoglu. "Pixel Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2016. arXiv: 1601.06759 [cs.CV].

[52] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. "Neural Discrete Representation Learning". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017. arXiv: 1711.00937 [cs.LG].

[53] A. van den Oord et al. *WaveNet: A Generative Model for Raw Audio*. 2016. arXiv: 1609.03499 [cs.SD].

[54] A. Orvieto et al. "Resurrecting Recurrent Neural Networks for Long Sequences". In: *International Conference on Machine Learning (ICML)*. 2023. arXiv: 2303.06349 [cs.LG].

[55] A. Radford et al. "Learning Transferable Visual Models from Natural Language Supervision". In: *International Conference on Machine Learning (ICML)*. 2021. arXiv: 2103.00020 [cs.CV].

[56] P. Ramachandran, B. Zoph, and Q. V. Le. *Searching for Activation Functions*. 2017. arXiv: 1710.05941 [cs.NE].

[57] F. Rosenblatt. "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6 (1958), pp. 386–408. DOI: 10.1037/h0042519.

[58] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. "Learning Representations by Back-Propagating Errors". In: *Nature* 323.6088 (1986), pp. 533–536. DOI: 10.1038/323533a0.

[59] T. Salimans and J. Ho. "Progressive Distillation for Fast Sampling of Diffusion Models". In: *International Conference on Learning Representations (ICLR)*. 2022. arXiv: 2202.00512 [cs.LG].

[60] T. Schaul, D. Borsa, J. Modayil, and R. Pascanu. *Ray Interference: A Source of Plateaus in Deep Reinforcement Learning*. 2019. arXiv: 1904.11455 [cs.LG].

[61] J. Schmidhuber. *Who Invented Backpropagation?* Linked from the lecture slide. 2014. URL: https://people.idsia.ch/~juergen/who-invented-backpropagation.html.

[62] Z. Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. Cited on the deck as the GRPO reference. 2024. arXiv: 2402.03300 [cs.CL].

[63] N. Shazeer. *GLU Variants Improve Transformer*. 2020. arXiv: 2002.05202 [cs.LG].

[64] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever. "Consistency Models". In: *International Conference on Machine Learning (ICML)*. 2023. arXiv: 2303.01469 [cs.LG].

[65] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.

[66] M. Toneva, A. Sordoni, R. Tachet des Combes, A. Trischler, Y. Bengio, and G. J. Gordon. "An Empirical Study of Example Forgetting during Deep Neural Network Learning". In: *International Conference on Learning Representations (ICLR)*. 2019. arXiv: 1812.05159 [cs.LG].

[67] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. "Graph Attention Networks". In: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1710.10903 [stat.ML].

[68] P. Vincent. "A Connection Between Score Matching and Denoising Autoencoders". In: *Neural Computation* 23.7 (2011). Dated 2010 on the lecture slide, pp. 1661–1674. DOI: 10.1162/NECO_a_00142.

[69] Z. Wang et al. "TacticAI: An AI Assistant for Football Tactics". In: *Nature Communications* 15 (2024), p. 1906. DOI: 10.1038/s41467-024-45965-x.

[70] P. J. Werbos. "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences". PhD thesis. Harvard University, 1974.

[71] D. L. K. Yamins, H. Hong, C. F. Cadieu, E. A. Solomon, D. Seibert, and J. J. DiCarlo. "Performance-Optimized Hierarchical Models Predict Neural Responses in Higher Visual Cortex". In: *Proceedings of the National Academy of Sciences* 111.23 (2014), pp. 8619–8624. DOI: 10.1073/pnas.1403112111.

[72] S. Yu et al. *Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think*. 2024. arXiv: 2410.06940 [cs.CV].

[73] T. Zaslavsky. "Facing Up to Arrangements: Face-Count Formulas for Partitions of Space by Hyperplanes". In: *Memoirs of the American Mathematical Society* 1.154 (1975). DOI: 10.1090/memo/0154.

[74] L. H. Zhang, V. Tozzo, J. M. Higgins, and R. Ranganath. "Set Norm and Equivariant Skip Connections: Putting the Deep in Deep Sets". In: *International Conference on Machine Learning (ICML)*. 2022. arXiv: 2206.11925 [cs.LG].

[75] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba. "Object Detectors Emerge in Deep Scene CNNs". In: *International Conference on Learning Representations (ICLR)*. 2015. arXiv: 1412.6856 [cs.CV].